VNUHCM – UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

COURSE: **PROGRAMMING TECHNIQUES**

MATCHING GAME PROJECT

Members:

1. Nguyễn Ngọc Hưng Phát – 23127099
2. Nguyễn Thanh Owen – 23127447

Instructor: Bùi Huy Thông

TABLE OF CONTENTS

I. Introduction.....5
II. Members Information.....5
III. Game Tutorial & Executing.....6
1. Game Executing.....6
2. Game Tutorial.....8
IV. Project Format Explanation.....11
1. Header Files.....11
1.1. GAME&MATCHING.....11
1.2. GRAPHICS.....11
1.3. LOGIN.....11
1.4. MENU.....11
1.5. STARTGAME.....12
1.6. STRUCT.....12
2. Cpp Files.....12
2.1. GAME&MATCHING.....12
2.2. GRAPHICS.....12
2.3. LOGIN.....12
2.4. MENU.....12
2.5. STARTGAME.....12
2.6. MAIN.....12
3. Text Files.....12
4. Sound Files.....13
5. Libraries.....14
V. Coding Explanations.....15
1. Standard Features.....15
1.1. Start.....15
1.2. Login.....16
1.3. Register.....17
1.4. Menu.....18

1.5. Generate 2-D Matrix 24
1.6. Print 2-D Matrix 24
1.7. Random Matrix 24
1.8. I Matching 25
1.9. Line Check 26
1.10. L Matching 27
1.11. U Matching 28
1.12. Z Matching 31
1.13. Check Matching 33
1.14. Check Existing Pairs 33
1.15. Reset Value 34
1.16. Movement 34
1.17. Random Position 35
1.18. Play 35
1.19. Save Records 40
1.20. Structure 40
2. Advanced Features 40
2.1. Color Effects 40
2.2. Visual Effects 43
2.3. Sound Effects 45
2.4. Background 46
2.5. Leaderboard 47
2.6. Move Suggestion 47
2.7. Stage Difficulty Increase 48
2.8. Game Finish Verify (Win) 48
2.9. Game Finish Verify (Lose) 49
2.10. Score Board 49
2.11. Help 51
3. Other Features 51
3.1. Forgot Password 51

3.2. Hidden Mode 52
3.3. Custom Level 53
VI. Demonstration Video 57
VII. Overall 57
VIII. Table Of Figures 58
IX. References 60

I. *Introduction*

The Matching Game, which is basically known with “Pikachu Puzzle Game” name, consists of a board of several cells, each of which presents for a figure. User will have to find and match a pair of cells that has the same figure and connect each other in some specific patterns. A correct match will make the two cells disappear. The game will end when all matching pairs are found.

In this project, we create a simplified version of this Matching Game by remaking the game with characters (instead of figures). We also create some new rules that make our version more interesting.

II. *Members Information*

1. Main Programmer: Nguyễn Thanh Owen – 23127447 – 23CLC10
   Owen takes the responsibility for checking the source code. Some functions were formed by both of us, but Owen mostly completed the ideas. Some functions that came from the ideas of Owen are: *start(), menu, UMatch, ZMatch, leaderboard, saveRecords, ...*
2. Report Writer and Supporter: Nguyễn Ngọc Hưng Phát – 23127099 – 23CLC10
   Phát is responsible for writing the report. Some functions that Phát completed: *PlaySound, checkAll, choose, …*
   Ideas from Phát which were completed by Owen: *Hidden mode, coefficient*.

III. *Game Tutorial & Executing*

1. *Game Executing*

At this section, we will introduce the way to setup our game file.

Firstly, you need to download the "23127099_23127447.zip". Right click to it and choose "Extract Here". Then, you will see a folder named "23127099_23127447".

Now, you need to do the following steps included in our video.
Here's the link to our video:
https://www.youtube.com/watch?v=9qYDrss8hH0

Reference: Lê Nguyên Nhật – 23127239 – 23CLC10 (how to setup MinGW file and .exe file)

Maybe you will find it hard to get the link to download MinGW, we will put the link in our video's description here:
https://drive.google.com/file/d/1ib0Nkb6QwkE1CySx1xiy5_-lvO5zBruB/view?usp=sharing
Enter this when you are in "cmd" screen:

*g++ -std=c++11 *.cpp -o PikachuGame –lWinmm*

Then, press *Enter* key to create .exe file.

After following the steps in our video, you will get an application file "PikachuGame".

Name	Date modified	Type	Size
PikachuGame	4/16/2024 12:56 AM	Application	72 KB
background	4/2/2024 12:03 AM	Text Document	1 KB
background2	3/31/2024 2:22 PM	Text Document	1 KB
board	4/2/2024 1:01 AM	Text Document	1 KB

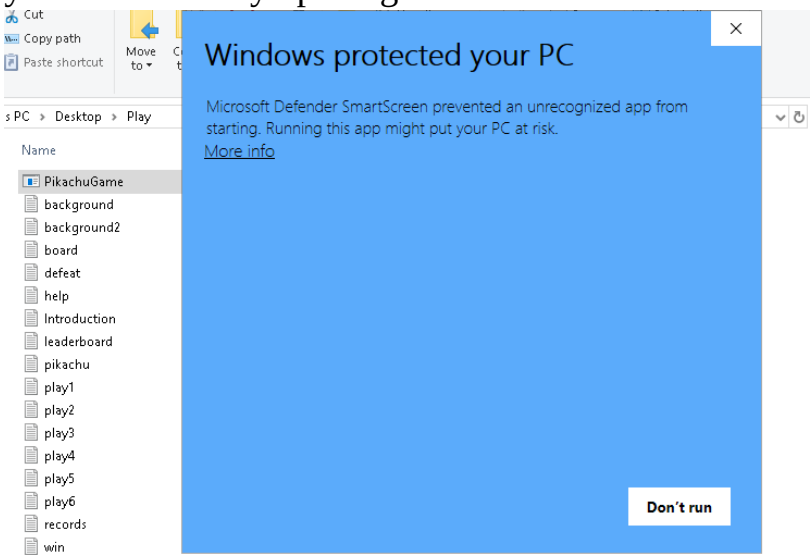
Picture 1 - Executing

At this time, you need to see our demonstration video to ensure that the game works correctly. Here's the link to our video:
https://www.youtube.com/watch?v=IPFroBZufyk

After having found the “PikachuGame” file, right click to the file, it will show a list of choices.

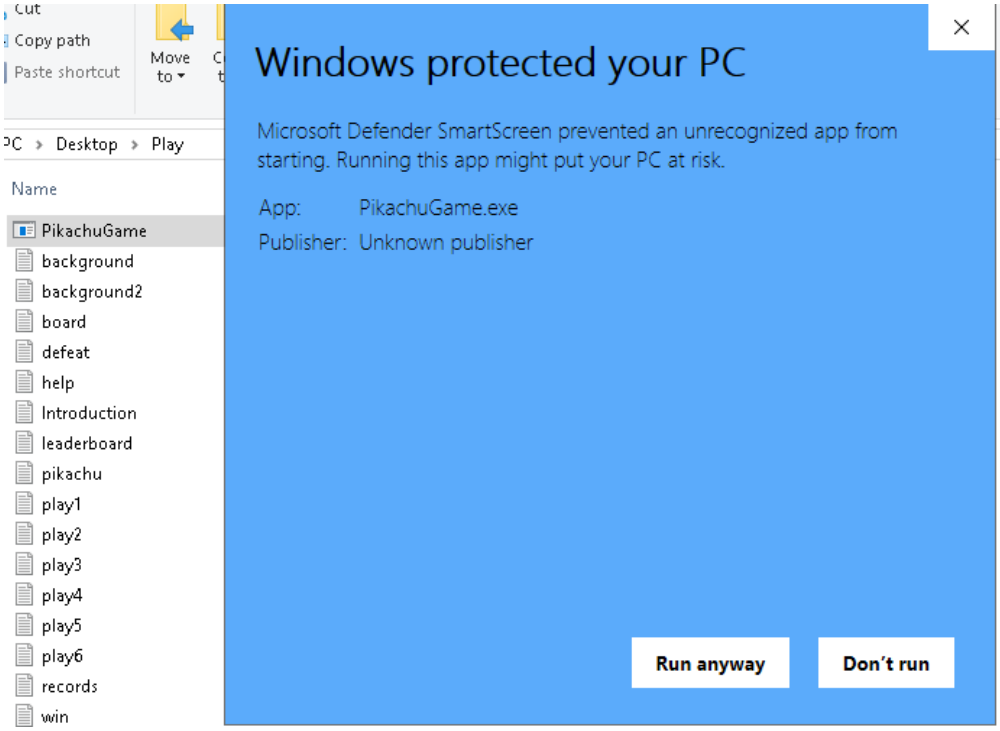
Picture 2 - Executing

Choose “Open” to open the file. Sometimes, your Windows will protect your PC when try opening these files.



Picture 3 - Executing

But don’t worry, our file doesn’t have any bad effect to your PC. Left click to “More Info”, then, choose “Run anyway”.

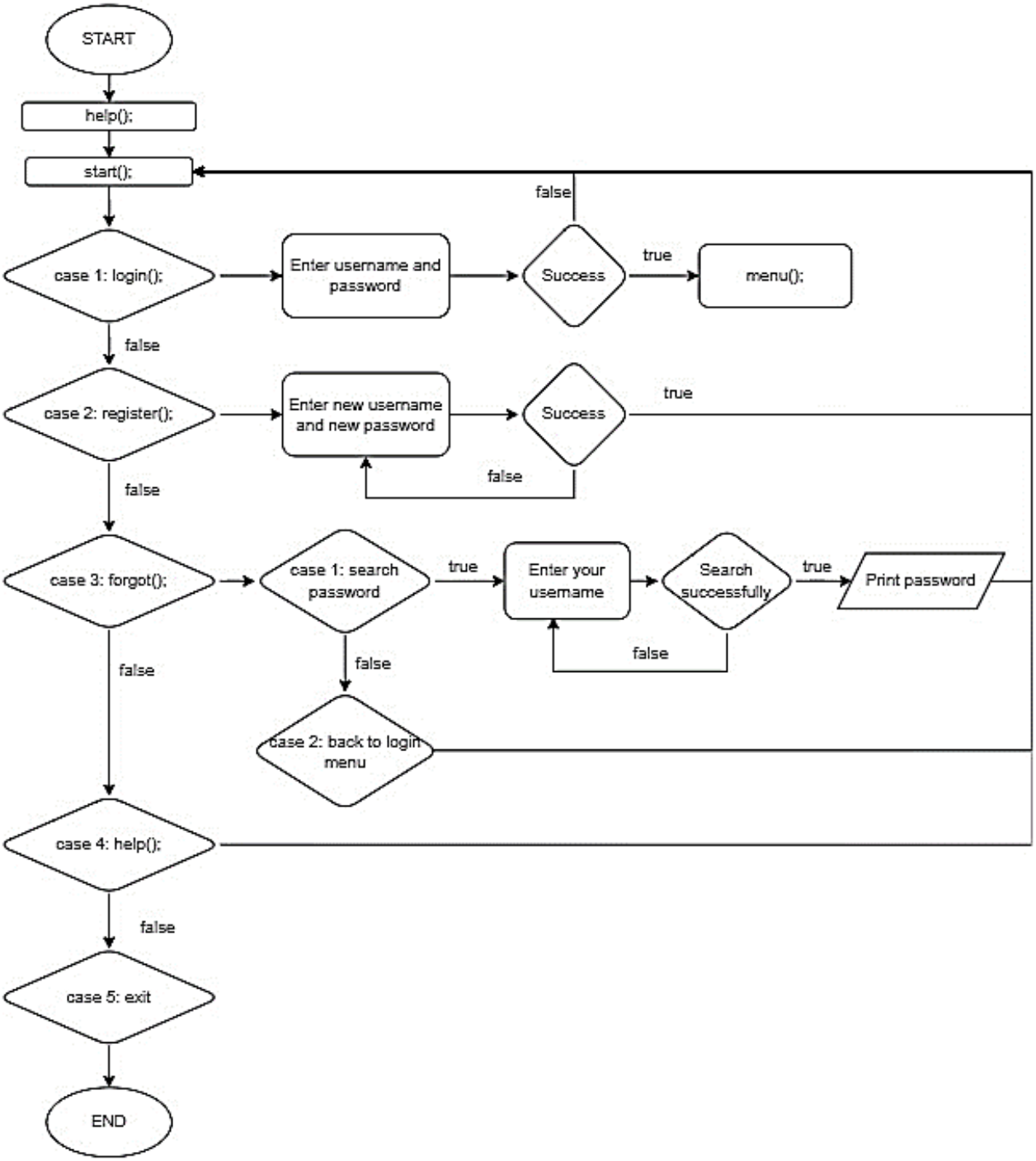


Picture 4 - Executing

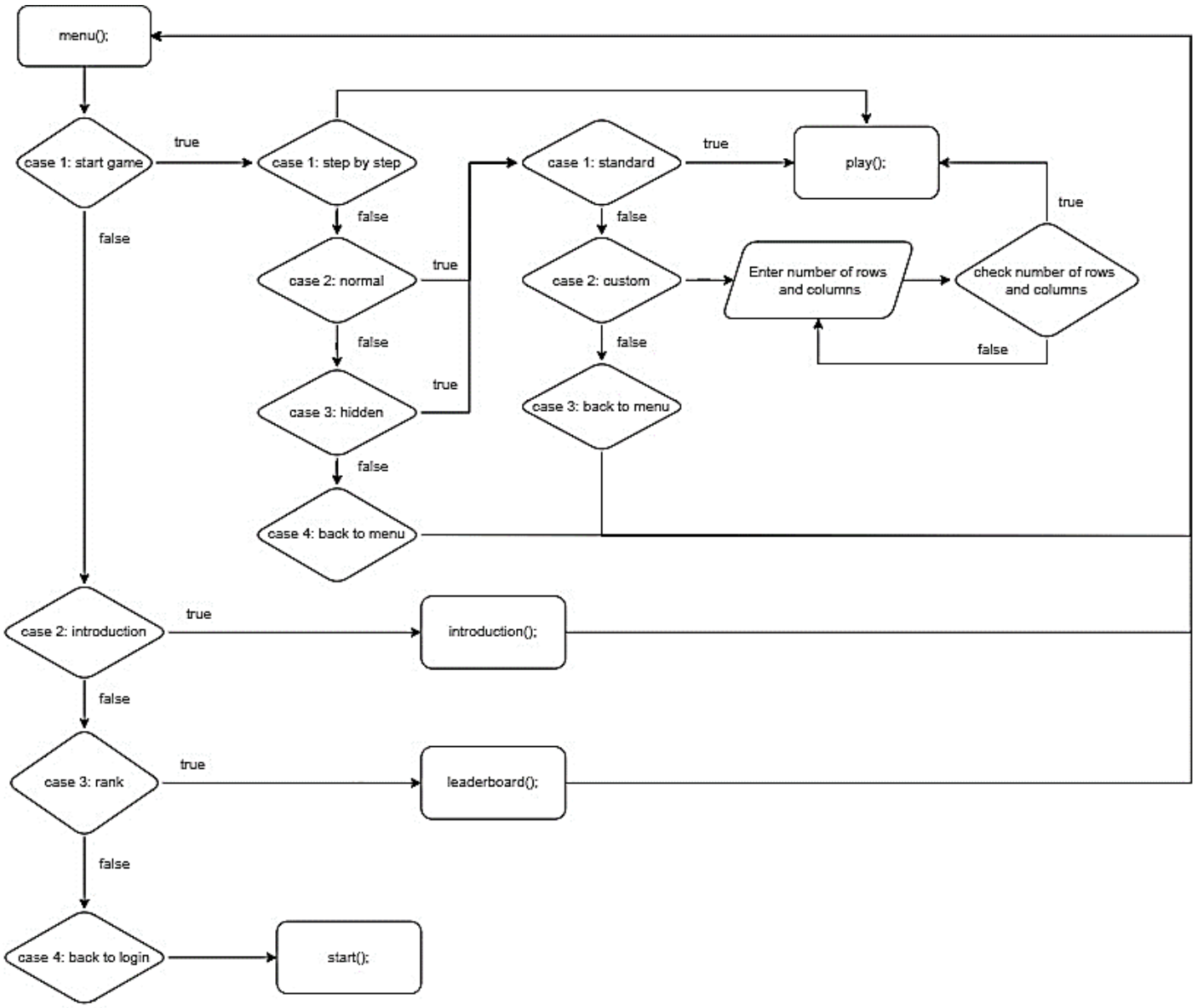
After doing all steps, this will be shown to you:

Picture 5 - Executing

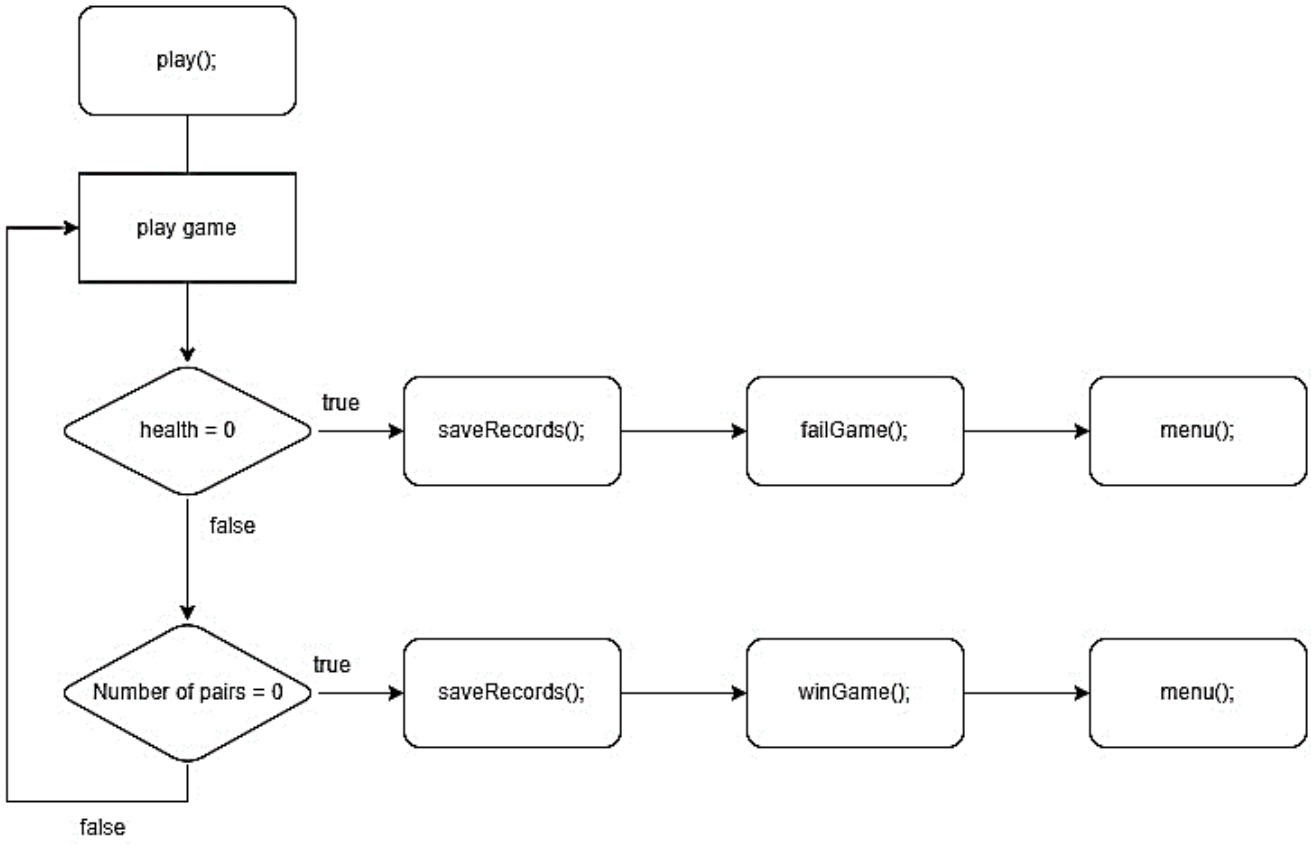
2. *Game Tutorial*



Picture 6 - Login



Picture 7 - Menu



Picture 8 - Play

IV. *Project Format Explanation*

1. *Header Files*

1.1. *GAME&MATCHING*

This header file contains the prototypes of functions used for checking the ability to match the pair of elements while playing. Besides, some functions are used for moving, helping player and generating the matrix again.

1.2. *GRAPHICS*

This header file contains the prototypes of almost the functions used for aesthetics. Most of them are used for drawing and colorizing. The last thing is that we define some buttons in this header file due to the fact that this header file is included in almost files in our program.

1.3. *LOGIN*

This header file is the one that checks the sign in process when the game is open. It contains some functions which ask player to create a new account or to sign in.

1.4. *MENU*

This header file holds the functions which are used in the MENU screen. This can be considered as the intermediate step before starting to play the game.

1.5. *STARTGAME*

The header file holds the prototypes of the functions used for playing the game. They will save the points of each player when the game ends.

1.6. *STRUCT*

The header file defines a structure of player's information including the account and points of each game mode.

2. *Cpp Files*

2.1. *GAME&MATCHING*

This cpp file will run the functions included in the header file "GAME&MATCHING".

2.2. *GRAPHICS*

This cpp file will run the functions included in the header file "GRAPHICS".

2.3. *LOGIN*

This cpp file will run the functions included in the header file "LOGIN".

2.4. *MENU*

This cpp file will run the functions included in the header file "MENU".

2.5. *STARTGAME*

This cpp file will run the functions included in the header file "STARTGAME".

2.6. *MAIN*

This cpp file is the first file to be run, and it calls some functions to start the game.

3. *Text Files*

Records.txt

This file is used to save players' accounts and their points of each game mode. This is the most important file, as without this file, we can't sign in so that we can't play the game.

Pikachu.txt

This file is used to decorate the start() screen.

Help.txt

This file is used to show the instructions how to play our game at first time.

Background.txt
Background2.txt
These files are used to decorate the menu screen.

Introduction.txt
This file is used to show some information of us and our teachers.

Leaderboard.txt
This file is used to show the ranks of players which arranged in ascending order. It plays an essential role since you can see your rank at that time, and even other players' ranks.

Play1.txt
Play2.txt
Play3.txt
Play4.txt
Play5.txt
Play6.txt
These files are used to draw a background behind the puzzle.

Board.txt
This file is used to draw the right-side board when playing which contains some necessary information in playing process.

Win.txt
Defeat.txt
These files are used to show a notification when the game ends.

4. *Sound Files*

GameStartMusic.wav
This sound file is used for playing the game music.

Navigation.wav
Enter.wav
These sound files are used for playing the sounds while moving the pointer and choosing an element.

CorrectMatch.wav
WrongMatch.wav
These sound files are used for playing the sounds when it checks if it's able to match the pair of elements or not.

Hint.wav
This sound file is used for playing a sound when player presses the defined key to see the move suggestion when playing.

WinGame.wav
GameOver.wav
These sound files are used for playing the sounds as a notification when the game ends.

5. *Libraries*

#include <iostream>
iostream is known as *input/output stream*. It's used for getting value from the keyboard by *cin* and printing contents to the monitor by *cout*.

#include <fstream>
fstream library is used for reading and writing files.

#include <windows.h>
windows.h library is used for calculating the size of the screen. Besides, our some drawing and colorizing functions won't work without this library.

#include <thread>
thread library is used when we need to pause the monitor for a while.

#include <string>
string library is used for create some variables in type *string*.

#include <time.h>
#include <stdlib.h>
These libraries are used to change the value of the elements randomly at current time.

#include <conio.h>

conio.h library is used for *_getch()* function which asks user to press a key on the keyboard.

```
#include <mmsystem.h>
#pragma comment(lib, "winmm.lib")
```

These are used for playing sound, *PlaySound* and *PlaySoundA* functions.

V. *Coding Explanations*

1. *Standard Features*

1.1. *Start*

start() function is used to call *login()*, *register()*, *help()*, *forgot()* functions.

Picture 9 - Start

Using up-arrow, down-arrow to move and *Enter* press key to choose which parts you want. Choose “Login” to call *login()* function (section 1.2.), “Register” to call *register()* function (section 1.3.), “Forgot

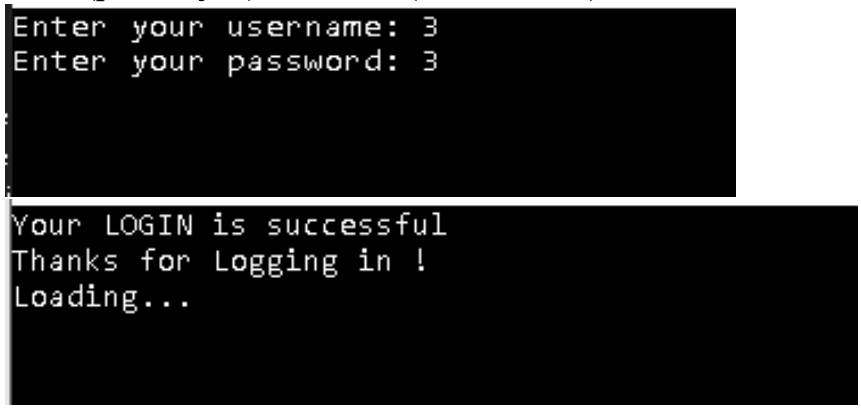
password” to call *forgot()* function (section 3.1.), “Help” to call *help()* function (section 2.11.), and “Exit” to exit the game.

1.2. *Login*

Reference:

https://www.youtube.com/watch?v=yo3ImzEwP50&list=PLVUI3PRPBHxU8IAvLI5yfOUtkI9Mwndgg&index=2

login() function is a member function of the *start()* function which calls the *login(), register(), help(), forgot()* functions.
Firstly, we created two Boolean variable *checkName* and *checkPass* that are used to check the user’s account, two *string* variables *username* and *password*, two *string* variables *name* and *pass* for reading file, a *string* variable *garbage* to get the rest data of the row that are not used for login process, and an *int* variable *posPlayer* to hold the position of the row while the file is being read.
Next, the user will be asked to enter *username* and *password*. Using *while* loop, the program will continuously read the “records.txt” file to get the data of *name* and *pass*, then it will ignore the rest data on each row, the *posPlayer* will increase by one each time it reads the next row, and it will stop reading when it reaches the end of file. If the *name* and *pass* that are read are the same as the *userName* and *password*, the two Boolean variables *checkName* and *checkPass* will get the *true* value.
If those *checkName* and *checkPass* are *true*, the program prints to the monitor a notification to show that it’s a successful login and calls the *menu(posPlayer)* function (section 1.4.).



Picture 10 – Login successfully

Otherwise, if the account can’t be found in the “records.txt” file after the *while* loop, it means that the account entered by the user isn’t

correct. Then, the program prints to the monitor a notification to tell the user that it's failed to login and calls the *start()* function again.

```
LOGIN ERROR
Please check your username and password
```

Picture 11 – Login unsuccessfully

1.3. *Register*

Reference:

https://www.youtube.com/watch?v=yo3ImzEwP50&list=PLVUI3PRPBHxU8IAvLI5yfOUtkI9Mwndgg&index=2

register() function is a member function of *start()* function which calls *login()*, *register()*, *help()*, *forgot()* functions.
Firstly, we created two *string* variables *ruserName* and *rpassword* that are used to register a new account. Next, the user will be asked to enter the *ruserName* and *rpassword*, the program will call the *checkRegistration("record.txt", ruserName, rpassword)* function (section 1.3.1.) to check whether the input *rusername* has already existed in the "records.txt" file or not. If the input *rusername* has already existed int the "records.txt", the *checkRegistration("record.txt", ruserName, rpassword)* will return *false* value, and return *true* value when the input *rusername* isn't found in the "record.txt" file.
Using the *while* loop, if the check function returns *false* value, the program will ask the user to enter *ruserName* and *rpassword* again.

```
Your username has existed, please enter again!
```

Picture 12 – Register unsuccessfully

Otherwise, the check function returns *true* value, it means that *rusername* can be used for registered, and the program will open the file "records.txt" in writing and appending mode to save the new *rusername* and *rpassword* to the file.

```
Registration is successfull!
```

Picture 13 – Register successfully

1.3.1. *Check Registration*

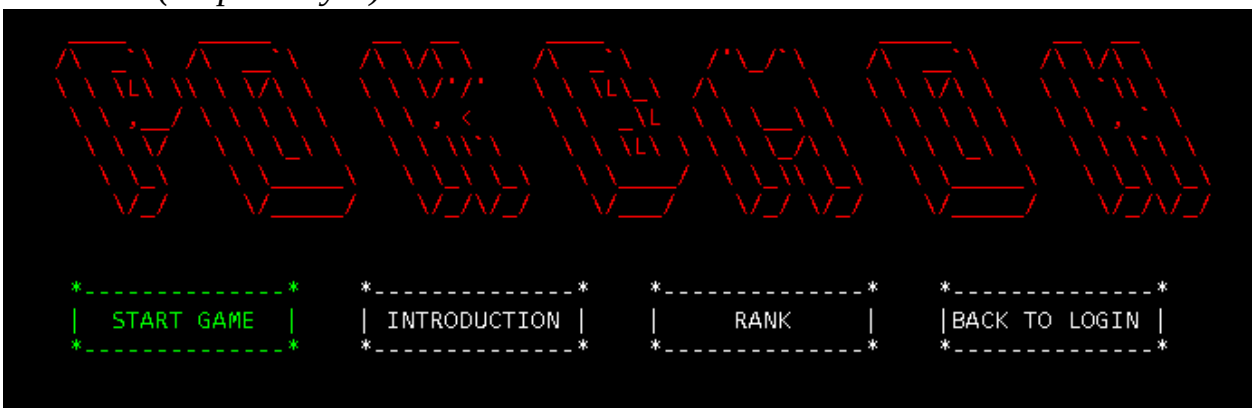
Reference:

https://www.youtube.com/watch?v=yo3ImzEwP50&list=PLVUI3PRPBHxU8IAvLI5yfOUtkI9Mwndgg&index=2

checkRegistration(string file, string userName, string password) is a Boolean function that returns *false* value when *userName* is found in the *file*, return *true* value when *userName* is not found in the file after reading *file*.

1.4. *Menu*

void menu(int posPlayer)



Picture 14 – Menu screen

Firstly, we created an *int* variable *numOfPlayer* initialized with value 0. Next, the program will open the "records.txt" file for reading, using *while* loop to count the number of rows that is equivalent to the number of accounts existed in the file.

After that, the program allocate an array *player* of structure *profile* (section1.20.) with *numOfPlayer* elements dynamically. Then, it opens the "records.txt" file again and read all the data of each row to each element in the array *player*.

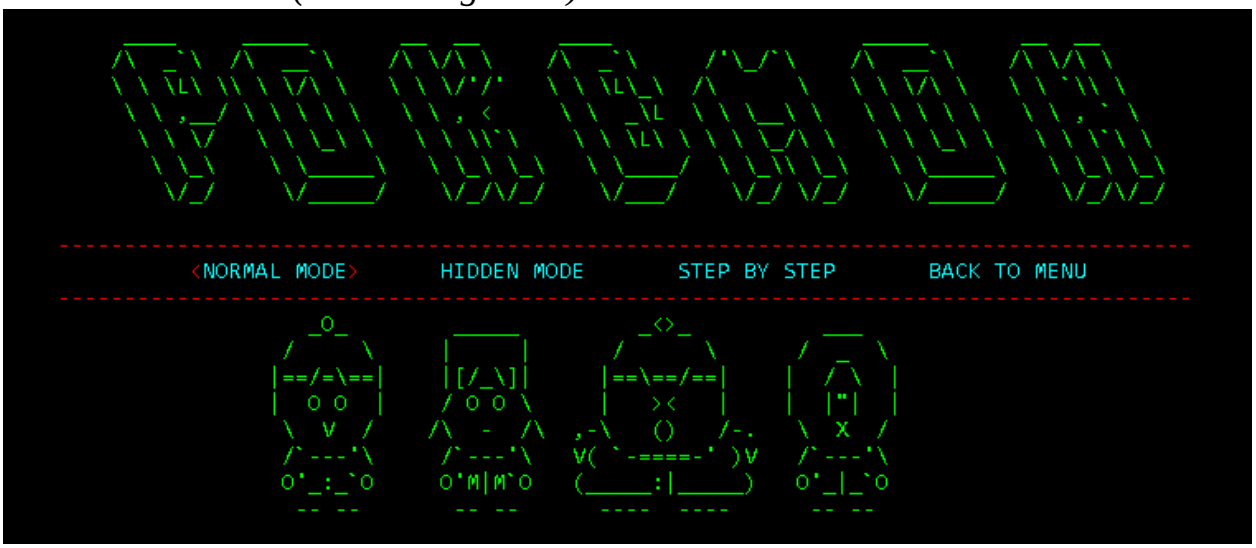
We created an array *s* of *string* type with four elements, an *int* variable *countString* to hold the index of each element in the array. When user presses *Enter* key, there will be four situations.

"INTRODUCTION": call *introduction()* function (section 1.4.1.) *(countString == 1)*.

"RANK": call *leaderboard(player, numOfPlayer, posPlayer)* function (section 2.5.) *(countString == 2)*.

"BACK TO LOGIN": call *start()* function *(countString == 3)*.

“START GAME”: *(countString == 0)*

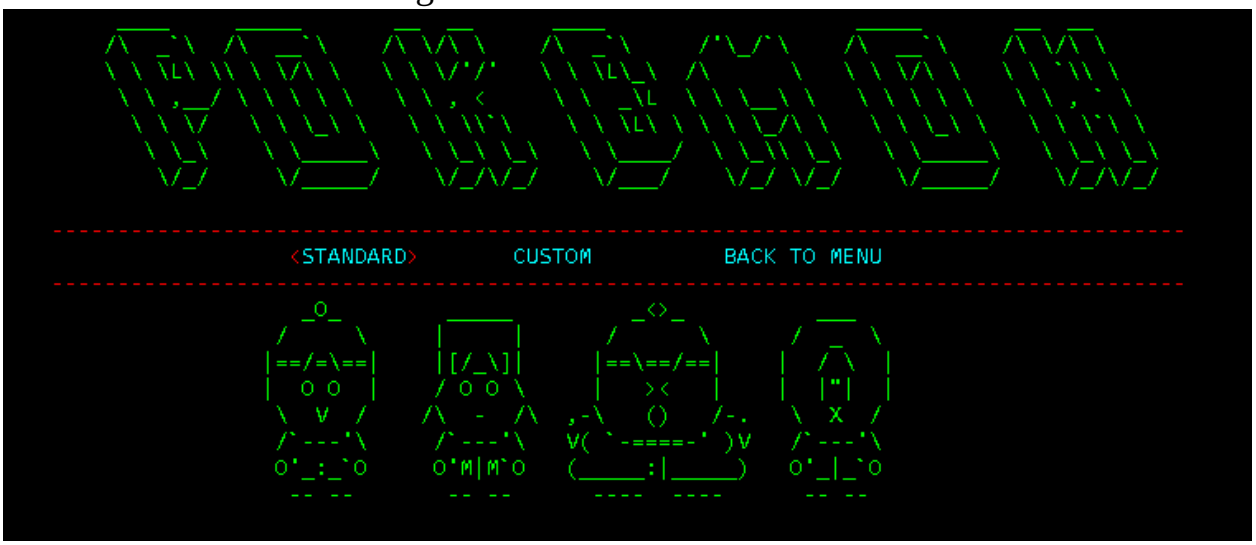


Picture 15 – Choose game mode

We created an array *mode* of *string* type with four elements to hold the choices of modes for the game, an *int* variable *posMode* to hold the index of element in *mode* array, an array *level* of *string* type with three elements to hold the choices of levels for the game, an *int* variable *posLevel* to hold the index of element in *level* array.

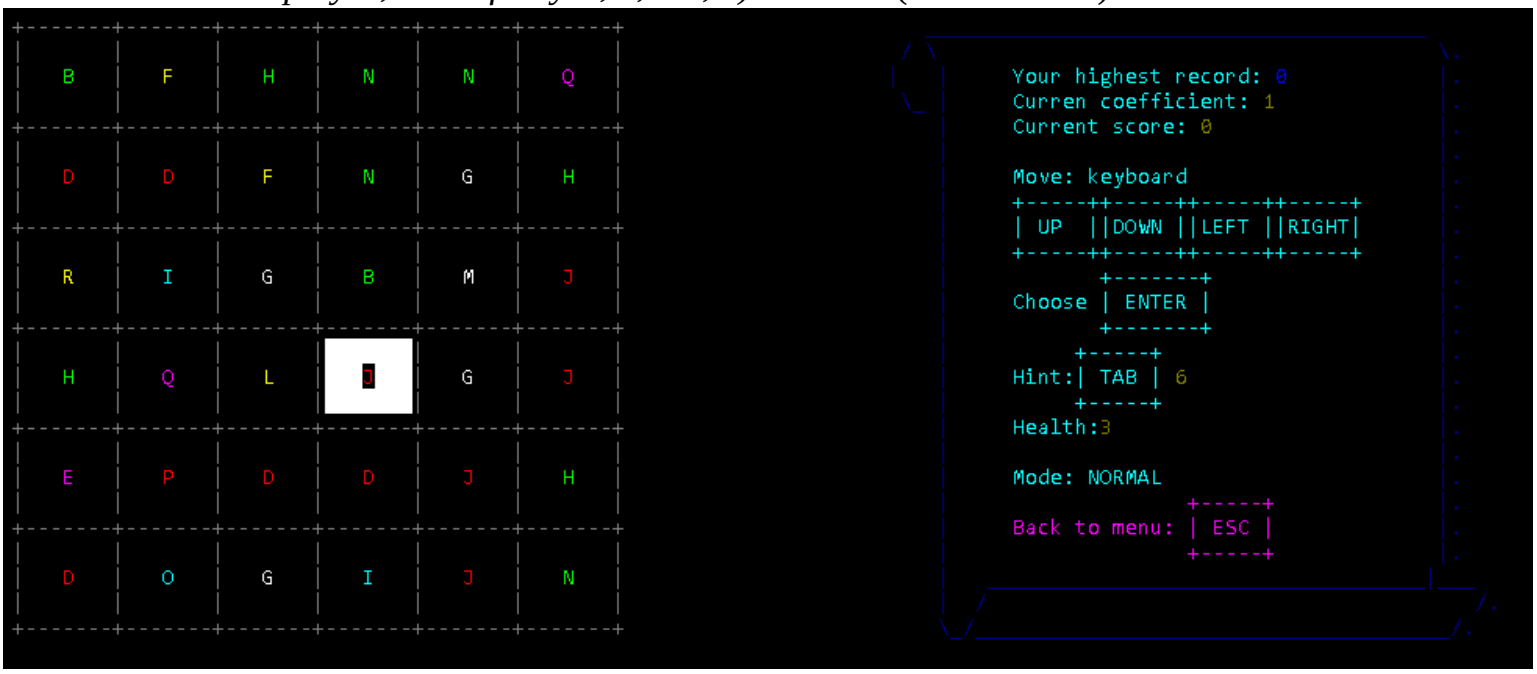
The program will call the *choose(mode,4,posMode)* function (section 1.4.1.) for user to choose the mode for the game. Using left-arrow, right-arrow keys to move, *Enter* press key to choose the options. If user presses *Enter* key to choose “BACK TO MENU” *(posMode = 3)*, it will call the *menu(posPlayer)* function again.
If user presses the *Enter* key to choose “STEP BY STEP” instead of “NORMAL MODE” or “HIDDEN MODE”, the program will immediately call the *play(4, 4, 0, posPlayer, player, numOfPlayer, 0, 1.0, 1)* function (section 1.18.).
Otherwise, if user presses *Enter* key at “NORMAL MODE” or “HIDDEN MODE”, the program will call *choose(level,3,posLevel)* for

user to choose level for the game.



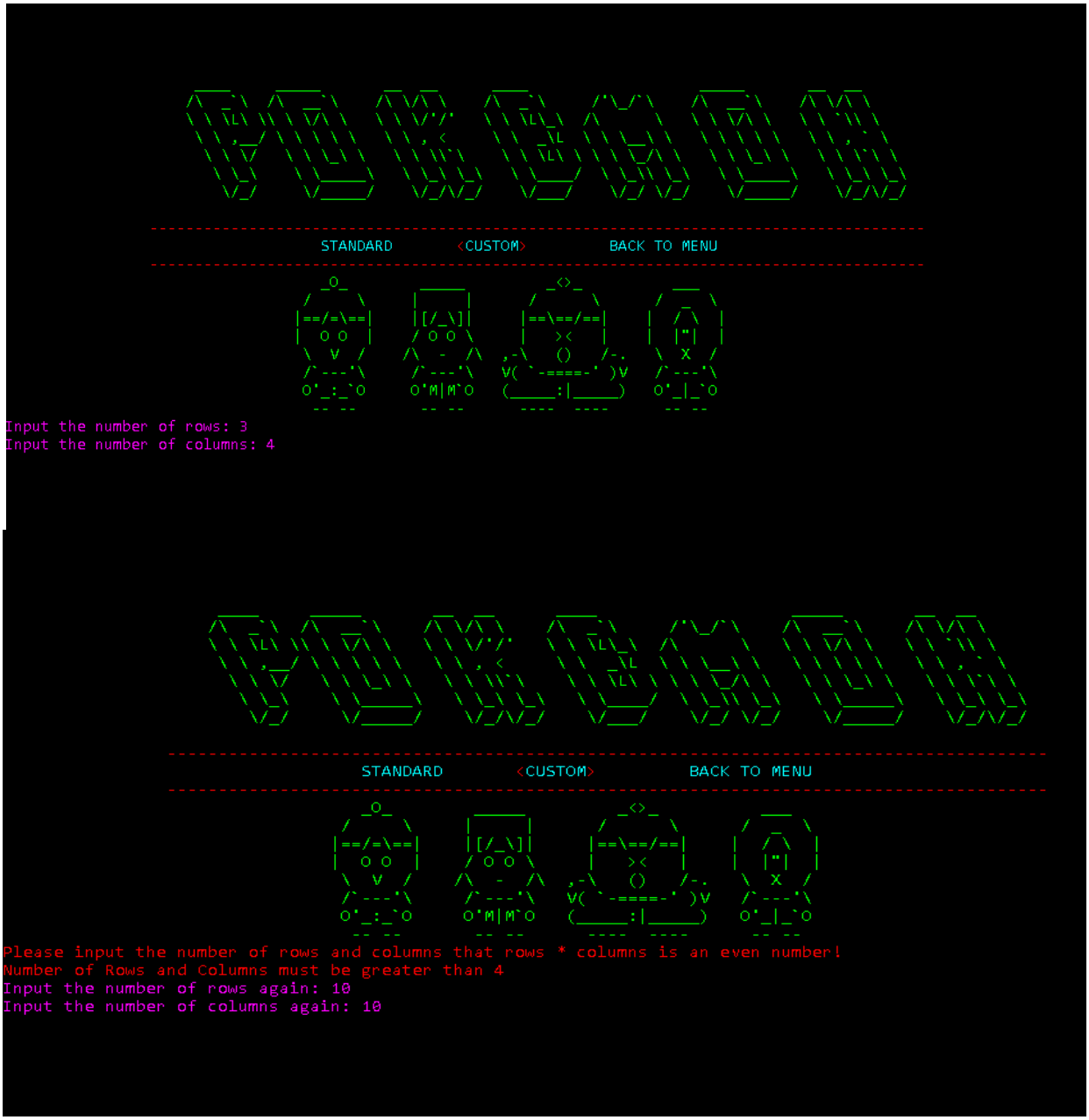
Picture 16 – Choose level

If user presses the *Enter* key to choose “BACK TO MENU” *(posLevel = 2)*, the program will call the *menu(posPlayer)* again.
If it’s not “BACK TO MENU”, there are some choices:
posMode = 0, *posLevel = 0*: which means user chose “NORMAL MODE” and “STANDARD”, then it calls *play(6, 6, 0, posPlayer, player, numOfPlayer, 0, 1.0, 0)* function (section 1.18.).



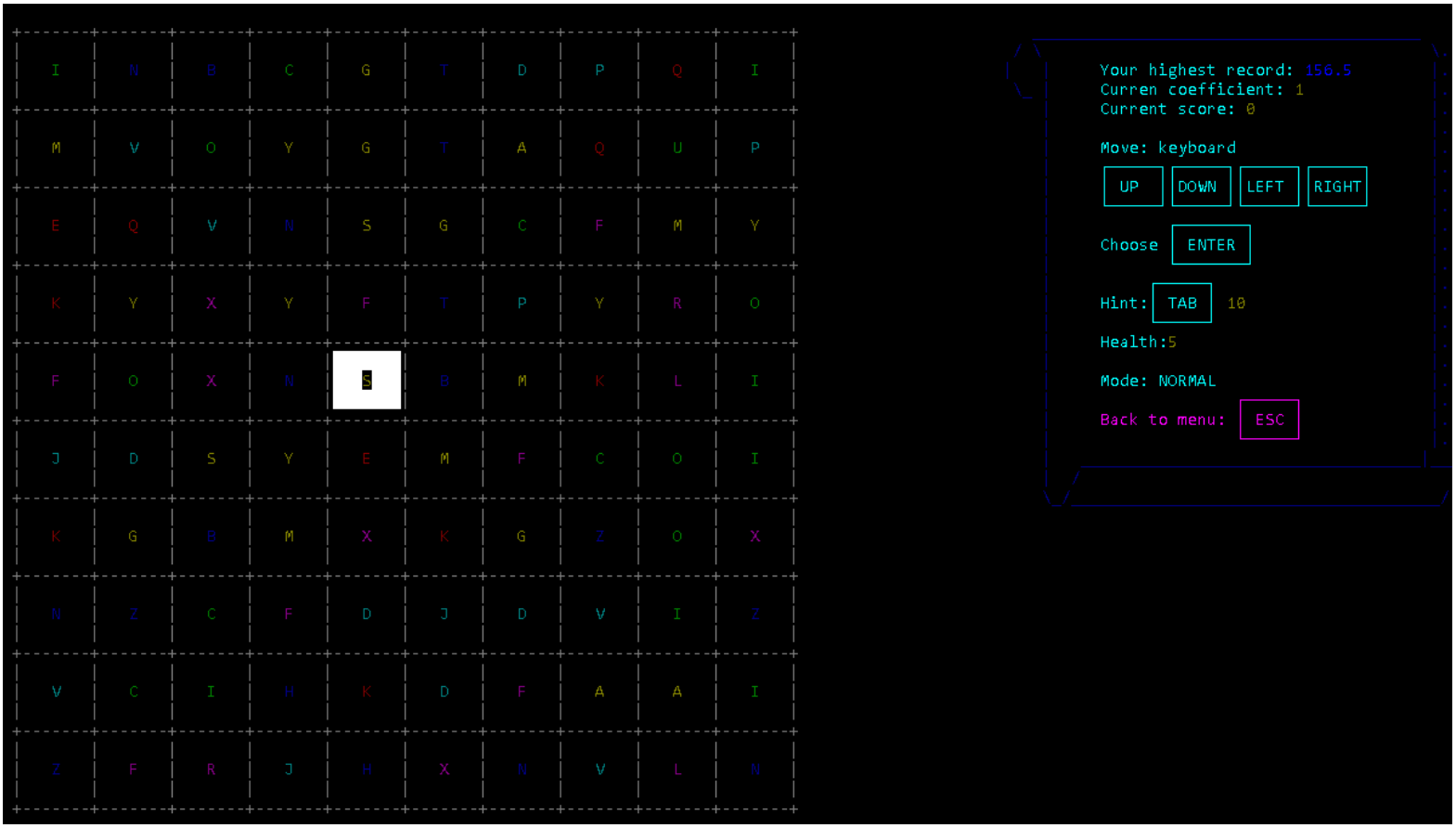
Picture 17 – Normal Standard

posMode = 0, *posLevel = 1*: which means user chose “NORMAL MODE” and “CUSTOM”, then the program creates two *int* variables *p*, *q* that stand for the number of rows and columns of the game. User will be asked to input the number of rows and columns as desired. *p* and *q* must be from 4 to 10. If they do not satisfy the conditions, the program will ask user to input the number of rows and columns again as we want our game not to be too small or too large.



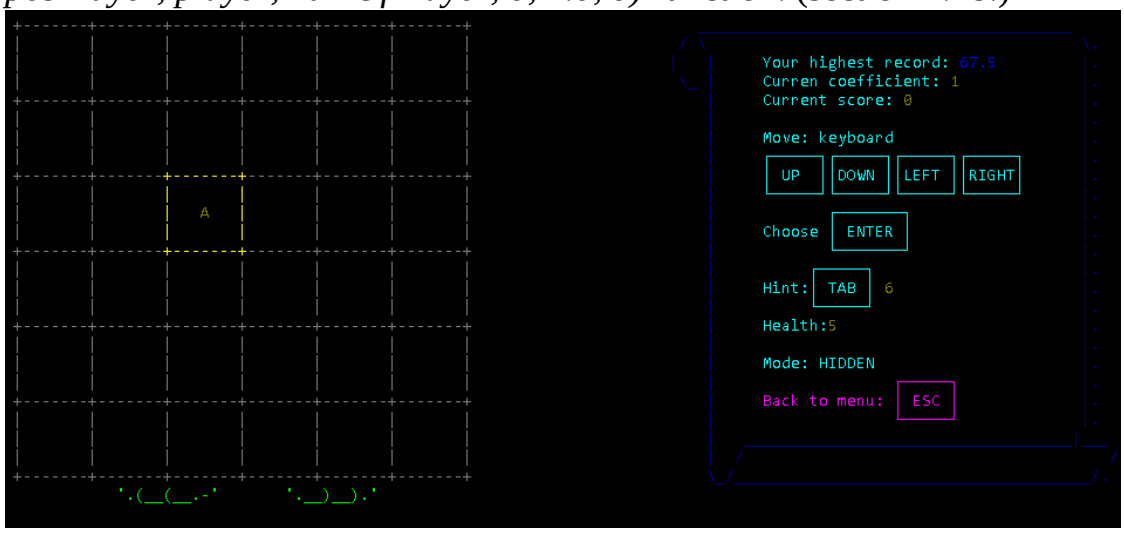
Picture 18 – Custom

Finally, it calls *play(p, q, 0, posPlayer, player, numOfPlayer, 0, 1.0, 0)* function. (section 1.18.)



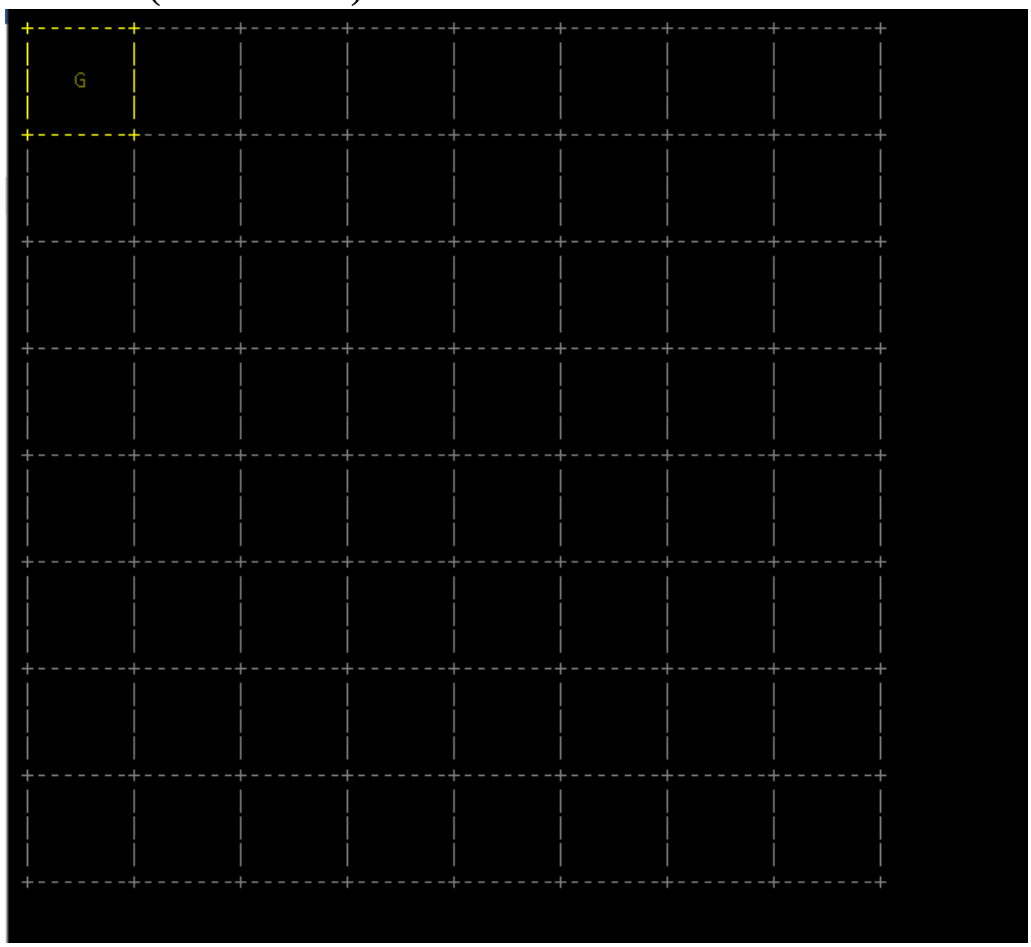
Picture 19 – Normal Custom

posMode = 1, *posLevel = 0*: which means user chose "HIDDEN MODE" (section 3.2.) and "STANDARD", then it calls *play(6, 6, 1, posPlayer, player, numOfPlayer, 0, 1.0, 0)* function. (section 1.18.)



Picture 20 - Hidden

posMode = 1, *posLevel = 1*: which means user chose “HIDDEN MODE” (section 3.2.) and “CUSTOM”, then the program creates two *int* variables *p, q* that stand for the number of rows and columns of the game. User will be asked to input the number of rows and columns as desired. *p* and *q* must be from 4 to 10. If they do not satisfy the conditions, the program will ask user to input the number of rows and columns again as we want our game not to be too small or too large. Finally, it calls *play(p, q, 1, posPlayer, player, numOfPlayer, 0, 1.0, 0)* function. (section 1.18.).



Picture 21 – Hidden Custom

1.4.1. *Choose*

choose(string ans[], int n, int pos) is used to change the pointer of the elements of *mode* array (section 1.4.) and *level* array (section 1.4.).

1.4.2. *Introduction*

void introduction(int posPlayer)

This function is just used to show our names and our teachers' names.

Picture 22 - Introduction

1.5. *Generate 2-D Matrix*

*char** gen2d(int p, int q)* is the function that will allocate a 2-D array *a* dynamically with *p* rows and *q* columns. Then, the program will call the *randomAgain(a,p,q)* function (section 1.7) to input the value to each elements of *a*. Finally, the function returns 2-D array *a*.

1.6. *Print 2-D Matrix*

*print2d(char** a, int p, int q, int mode, int indexBackground)* is a function to print the matrix *a* to the monitor, depends on which *mode* it is. Before printing the matrix, it will call *backgroundPlay(indexBackground)* function to print the background, then, using two *for* loops to print each element which has the value is not a whitespace character till the end of *a*. Each time it prints an element, the program calls *box(2 + j * 8, 2 + i * 4, 4, 8, a[i][j], mode, 8)* function (section 2.1.1.) to draw the box around the value.

1.7. *Random Matrix*

Firstly, we created an *int* variable *amount* to hold the number of pairs of elements that still exist in the matrix. Then, the program calls *countNumOfEles(a, p, q)* function (section 1.7.1) to calculate the number of elements that are not whitespace character. *amount* =

countNumOfEles(a, p, q) / 2 that means now, *amount* will hold the number of pairs of elements that are different from whitespace character. If *amount* is greater than 26, then it sets *amount = 26*, as we want our matrix just has the value from A – Z for each elements. Finally, the program change the value of each elements (excluded the elements which have the whitespace character value) randomly from A – Z by using *65 + rand() % amount*.

1.7.1. Counting Numbers Of Elements

We created an *int* variable *count* to count the number of elements in the matrix that still have the value differs from whitespace character.

*countNumOfEles(char** a, int p, int q)* is a function that uses two *for* loops to scan all the matrix *a*, whenever it finds an element whose value is different from whitespace character, *count* will be increased by one

Finally, the function returns *count* as the number of elements that satisfied the requirements.

1.8. *I Matching*

*IMatch(char** a, int posRow1, int posCol1, int posRow2, int posCol2)*

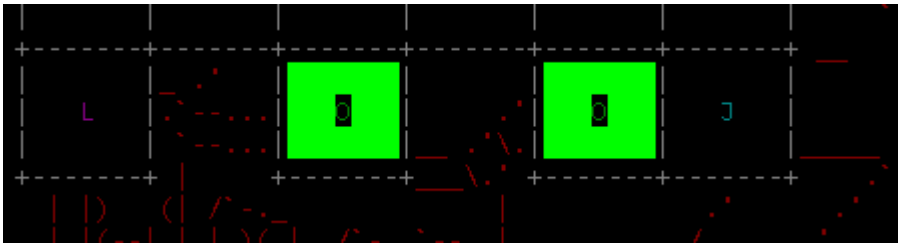
Reference:

https://github.com/PhVanMin/Pikachuuu/blob/master/Check1.cpp

The *IMatch* function is used to check if there is an "I" way that connects two elements at *[posRow1][posCol1]* and *[posRow2][posCol2]*.

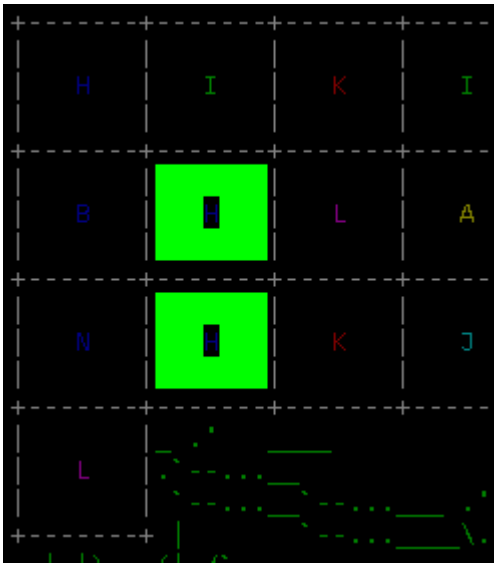
There are two situations:

The two elements have the same index of row *(posRow1 == posRow2)*: We created two *int* variables *x*, *y* that hold the value of *posCol1* and *posCol2*. Using condition, *x* will hold the smaller value, whereas *y* holds the greater value. If *posCol1* is as same as *posCol2*, *x* and *y* certainly holds the same value. Using a *for* loop to scan from the column that next to *x* to the column right before *y*, if there is any element in the scan process whose value is different from whitespace character, it means that there is an element that interrupts the way from *x* to *y* and it will return *false* value immediately. Otherwise, it returns *true* value.



Picture 23 – I Matching

The two elements have the same index of column *(posCol1 == posCol2):*
We created two *int* variables *x*, *y* that hold the value of *posRow1* and *posRow2*. Using condition, *x* will hold the smaller value, whereas *y* holds the greater value. If *posRow1* is as same as *posRow2*, *x* and *y* certainly holds the same value. Using a *for* loop to scan from the column that next to *x* to the column right before *y*, if there is any element in the scan process whose value is different from whitespace character, it means that there is an element that interrupts the way from *x* to *y* and it will return *false* value immediately. Otherwise, it returns *true* value.



Picture 24 – I Matching

1.9. *Line Check*

*bool lineCheck(char**a, int posRow1, int posCol1, int posRow2, int posCol2)*

Reference:

https://github.com/PhVanMin/Pikachuuu/blob/master/Check1.cpp

Pseudo Codes:

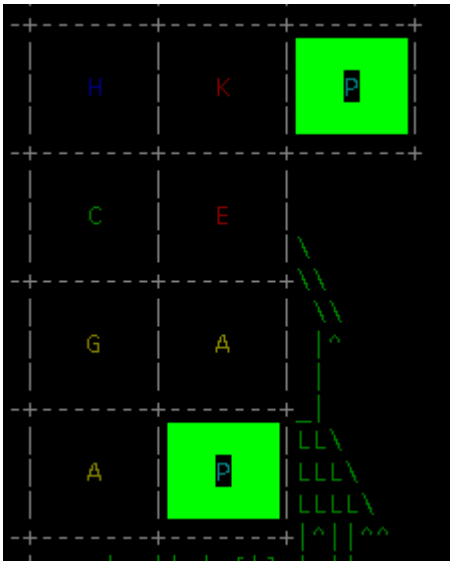
```
if (posRow1 is not equal to posRow2) and (posCol1 is not equal to posCol2)
   return false;

if posRow1 is equal to posRow2:
   x = min(posCol1, posCol2)
   y = max(posCol1, posCol2)
   for i = x to y:
         if a[posRow1][i] is not equal to 32:
               return false

   return true


   if posCol1 is equal to posCol2:

         x = min(posRow1, posRow2)

         y = max(posRow1, posRow2)

   for i = x to y:
         if a[i][posCol2] is not equal to 32:

               return false

   return true
```

1.10. *L Matching*

*bool LMatch(char** a, int posRow1, int posCol1, int posRow2, int posCol2)*

Reference:

https://github.com/PhVanMin/Pikachuuu/blob/master/Check1.cpp



Picture 25 – L Matching

Pseudo Code:

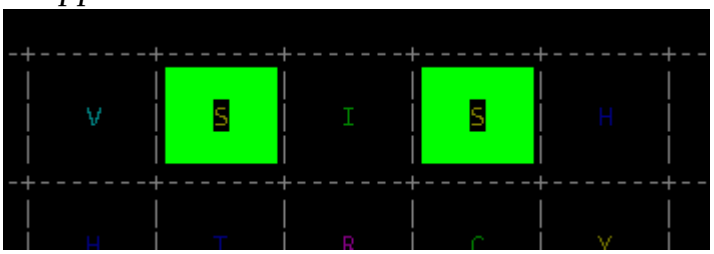
```
if posRow1 is equal to posRow2 or posCol1 is equal to posCol2:
   return false

check1 = false
check2 = false

if a[posRow1][posCol2] is equal to ' ':
   check1 = IMatch(a, posRow1, posCol1, posRow1, posCol2)
   check2 = IMatch(a, posRow2, posCol2, posRow1, posCol2)
            check1 and check2:
                     return true
if a[posRow2][posCol1] is equal to ' ':
   check1 = IMatch(a, posRow1, posCol1, posRow2, posCol1)
   check2 = IMatch(a, posRow2, posCol2, posRow2, posCol1)
            if check1 and check2:
                     return true


return false
```

1.11. *U Matching*

*bool UMatch(char** a, int p, int q, int posRow1, int posCol1, int posRow2, int posCol2)*

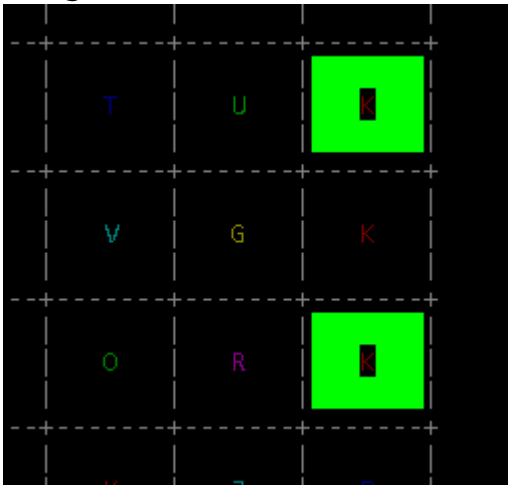
Pseudo Code:

```
if ((posRow1 == 0 && posRow2 == 0) || (posRow1 == p - 1 &&
posRow2 == p - 1))
   return true
if ((posCol1 == 0 && posCol2 == 0) || (posCol1 == q - 1 && posCol2
== q - 1))
   return true
// Upper U check
```



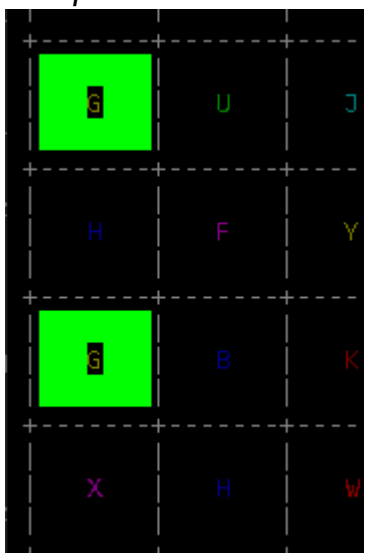
Picture 26 – Upper U Matching

```
up = min(posRow1, posRow2)

if (IMatch(a, posRow1, posCol1, -1, posCol1) && IMatch(a, posRow2,
posCol2, -1, posCol2))
   return true
else
   for i = up - 1 to 0 step -1
      if (IMatch(a, posRow1, posCol1, i - 1, posCol1) && IMatch(a,
posRow2, posCol2, i - 1, posCol2))
         if (IMatch(a, i, posCol1, i, posCol2))
            return true
// Right U check
```



Picture 27 – Right U Matching

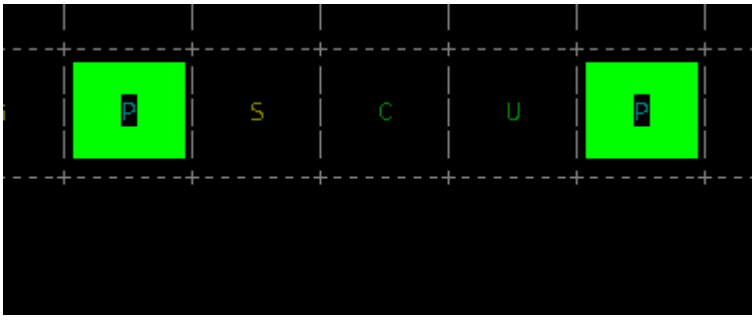
```
right = max(posCol1, posCol2)

if (IMatch(a, posRow1, posCol1, posRow1, q) && IMatch(a, posRow2,
posCol2, posRow2, q))
   return true
else
   for i = right + 1 to q - 1
      if (IMatch(a, posRow1, posCol1, posRow1, i + 1) && IMatch(a,
posRow2, posCol2, posRow2, i + 1))
         if (IMatch(a, posRow1, i, posRow2, i))
            return true
// Left U check
```



Picture 28 – Left U Matching

```
left = min(posCol1, posCol2)

if (IMatch(a, posRow1, posCol1, posRow1, -1) && IMatch(a,
posRow2, posCol2, posRow2, -1))
   return true
else
   for i = left - 1 to 0 step -1
      if (IMatch(a, posRow1, posCol1, posRow1, i - 1) && IMatch(a,
posRow2, posCol2, posRow2, i - 1))
         if (IMatch(a, posRow1, i, posRow2, i))
            return true
```

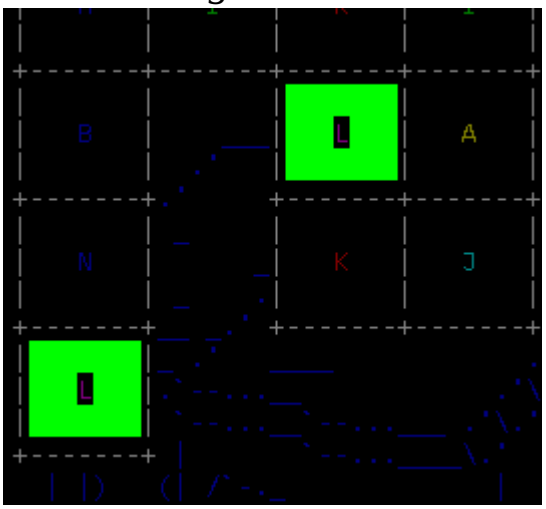
// *Down U check*



Picture 29 – Down U Matching

```
down = max(posRow1, posRow2)

if (IMatch(a, posRow1, posCol1, p, posCol1) && IMatch(a, posRow2,
posCol2, p, posCol2))
   return true
else
   for i = down + 1 to p - 1
      if (IMatch(a, posRow1, posCol1, i + 1, posCol1) && IMatch(a,
posRow2, posCol2, i + 1, posCol2))
         if (IMatch(a, i, posCol1, i, posCol2))
            return true

// When there's not any way to match
return false
```

1.12. *Z Matching*



Picture 30 – Z Matching

Pseudo Code:

```
if posRow1 is equal to posRow2 or posCol1 is equal to posCol2:
        return false
  if posCol1 < posCol2:
    x = posCol1
    y = posCol2
    for i = x + 1 to y - 1:
      if lineCheck(a, posRow1, i, posRow2, i):
        if lineCheck(a, posRow1, posCol1 + 1, posRow1, i)
       and lineCheck(a, posRow2, posCol2 - 1, posRow2, i):
          return true
  else:
    x = posCol2
    y = posCol1
    for i = x + 1 to y - 1:
      if lineCheck(a, posRow1, i, posRow2, i):
        if lineCheck(a, posRow1, posCol1 - 1, posRow1, i)
       and lineCheck(a, posRow2, posCol2 + 1, posRow2, i):
          return true


  if posRow1 < posRow2:
    x = posRow1
    y = posRow2
    for i = x + 1 to y - 1:
      if lineCheck(a, i, posCol1, i, posCol2):
        if lineCheck(a, posRow1 + 1, posCol1, i, posCol1)
       and lineCheck(a, posRow2 - 1, posCol2, i, posCol2):
```

```
            return true
    else:
        x = posRow2
        y = posRow1
        for i = x + 1 to y - 1:
            if lineCheck(a, i, posCol1, i, posCol2):
                if lineCheck(a, posRow1 - 1, posCol1, i, posCol1)
               and lineCheck(a, posRow2 + 1, posCol2, i, posCol2):
                    return true

    return false
```

1.13. *Check Matching*

*bool checkAll(char** a, int p, int q, int posRow1, int posCol1, int posRow2, int posCol2)*

If the value of two elements *a[posRow1][posCol1]* and *a[posRow2][posCol2]* are different, obviously, we don't have to check, the function returns *false* value immediately.

The function will sequentially call

IMatch(a, posRow1, posCol1, posRow2, posCol2)

LMatch(a, posRow1, posCol1, posRow2, posCol2)

ZMatch(a, posRow1, posCol1, posRow2, posCol2)

UMatch(a, p, q, posRow1, posCol1, posRow2, posCol2)

to check. If one of four check functions above has *true* value, it means that the two elements at *[posRow1][posCol1]* and *[posRow2][posCol2]* can be matched, then the *checkAll* function will return *true* value immediately.

Otherwise, after calling all the check functions and *checkAll* function still doesn't return *true* value, it means that the two elements at *[posRow1][posCol1]* and *[posRow2][posCol2]* can't be matched, then the *checkAll* function will return *false* value.

1.14. *Check Existing Pairs*

*bool checkExistPair(char** a, int p, int q)*

This function is used to check whether there's still a pair of elements that can be matched.
Using four *for* loops to check, two outer loops will scan from the beginning of the matrix *a* to the end of the matrix *a* for the first element, while two inner loops will scan from the element of the two outer loops to the end of the matrix *a*. Each time it scans, the program calls *checkAll* function to check whether the two elements can be matched or not.
If the *checkAll* function returns *true* value, it means that there's at least one pair of elements have not been matched yet, then the *checkExistPair* function will returns *true* value. Otherwise, after the four *for* loops, function *checkExistPair* still can't return *true* value, it means that there's no left pair of elements can be matched in the matrix *a*. Then, it returns *false* value.

1.15. *Reset Value*

*resetEle(char** a, int p, int q, int posRow1, int posCol1, int posRow2, int posCol2, int choice)*
At first, the program will call *checkAll(a, p, q, posRow1, posCol1, posRow2, posCol2)* to check whether the two elements *a[posRow1][posCol1]* and *a[posRow2][posCol2]* can be matched or not. If the *checkAll* function returns *true* value, then the program will change the *a[posRow1][posCol1]* and *a[posRow2][posCol2]* to whitespace character.
Changing the value to whitespace character in order to use the makes the *print2d* function not print the elements that contain whitespace character value.

1.16. *Movement*

1.16.1. *Move Up*

*moveUp(char** a, int p, int q, int& x, int& y)*
This function will move the pointer to a position above the current position if possible. If it is not possible, it will move through nearby columns one by one to compare.

1.16.2. *Move Down*

*moveDown(char** a, int p, int q, int& x, int& y)*
This function will move the pointer to a position below the current position if possible. If it is not possible, it will move through nearby columns one by one to compare.

1.16.3. *Move Left*

*moveLeft(char** a, int p, int q, int& x, int& y)*
This function will move the pointer to a position to the left of the current position if possible. If it is not possible, it will move through nearby rows one by one to compare.

1.16.4. *Move Right*

*moveRight(char** a, int p, int q, int& x, int& y)*
This function will move the pointer to a position to the right of the current position if possible. If it is not possible, it will move through nearby rows one by one to compare.

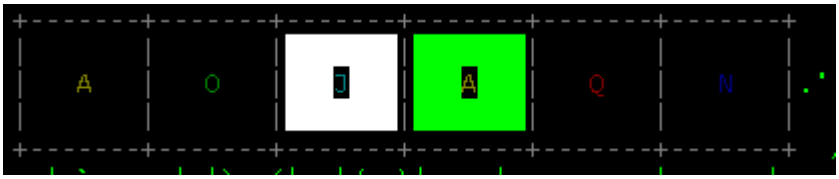
1.17. *Random Position*

*randomPos(char** a, int p, int q, int& x, int& y)*
After using *resetEle* function, the program needs to change the pointer to another place that still contain the value from A – Z. Using *while* loop, the program will change the position of the pointer randomly until it finds an element that has the value which is different from whitespace character.

1.18. *Play*

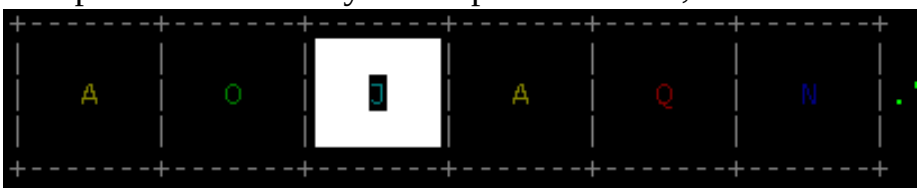
void play(int p, int q, int mode, int posPlayer, profile player, int numOfPlayer, double tempPoint, double coefficient, int stage)*

Firstly, the program creates an matrix with *p* rows and *q* columns. We created an *int* variables *health = 5*,two *int* variables *numOfHints = (p + q) / 2* and *numOfPairs = p * q / 2* , two *int* variables *x, y* to hold the latest pointer's index, two *int* variables *xcu, ycu* to hold the last pointer's index, an *int* variable *count* for counting the times that the *Enter* key is pressed, four *int* variables *row1, col1, row2, col2* to hold the index of the first and second elements when user presses the *Enter* key.
The matrix *a* will be printed by using *print2d* function, depends on which mode it is.
Using a *while* loop. Inside the *while* loop, if there's no pair of elements can be matched, it will call *randomAgain* function, and print all necessary things. When user presses arrow keys to move, it will calls the *moveUp*, *moveDown*, *moveLeft*, *moveRight* depends on which arrow key is pressed.
When user presses the *Enter* key, *x* and *y* will hold the index of the element that the pointer is at, and *thanhSang* function (section 2.2.3.) will colorize that element to make it differ from the others.



Picture 31 – Choose element

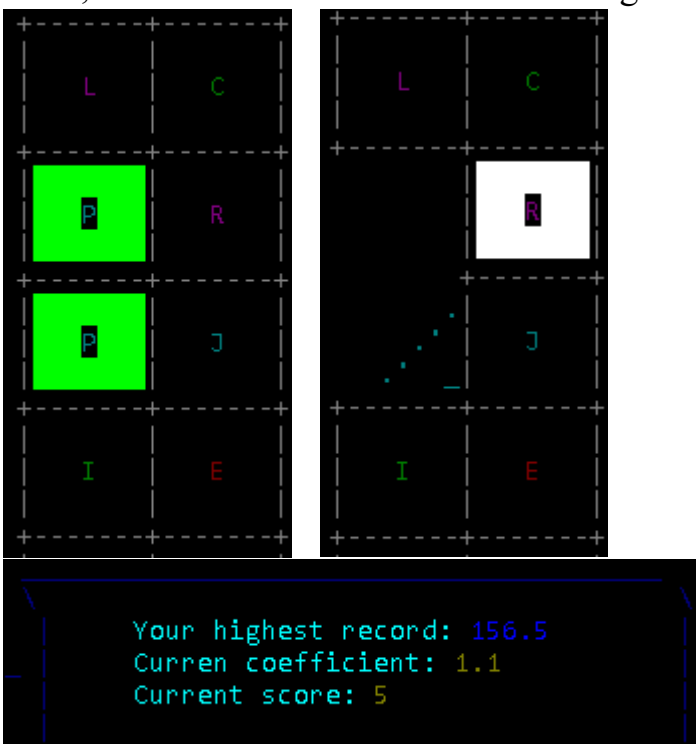
count will be increased by one each time user presses the *Enter* key, if user press the *Enter* key at one position twice, *count* will be set to 0.



Picture 32 – Cancel chosen element

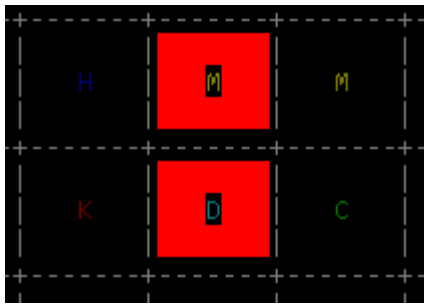
On the contrary, if user presses the *Enter* key two times at two different elements, the program will call the *checkAll* function that is to check whether these two elements can be matched or not. *count* will be set to 0 again.

If the *checkAll* function returns *true* value, *thanhSang* function (section 2.2.3.) will colorize two boxes containing these two elements into green color, which means it's a correct matching.



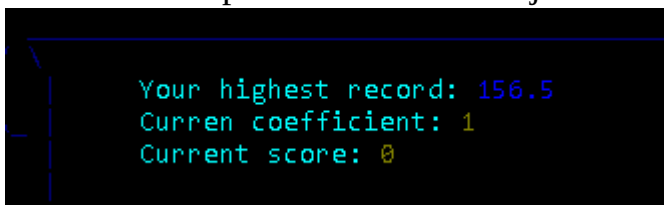
Picture 33 – Correct matching

Each time matching correctly, user will get 5**coefficient* points for *tempPoints*, *coefficient* is increased by 0.1, *numOfPairs* is decreased by 2, and it will play a happy sound to show that one pair of elements has just been matched correctly. After all, set *row1 = col1 = row2 = col2 = -1*, which means that *row1*, *col1*, *row2*, *col2* don't hold the index of any elements in the matrix *a*, and the program will update the recent *tempPoint*, *coefficient*, *numOfHints*, *health* to the right-side board. The program will call the *randomPos* to change the pointer randomly. Otherwise, if the *checkAll* function returns *false* value, *thanhSang* function (section 2.2.3.) will colorize two boxes containing these two elements into red color, which means it's an incorrect matching.



Picture 34 – Wrong matching

User's points will be decreased by 5 if the points are greater or equal to 5, *coefficient* will be set to 1.0, and it will play an unpleasant sound to show that one pair of elements has just been matched incorrectly.



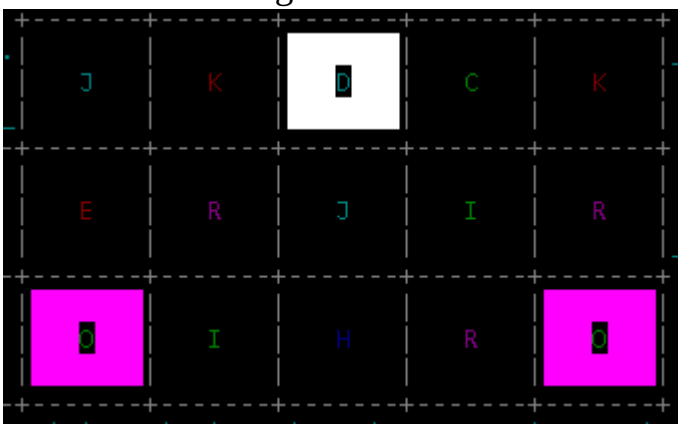
Picture 35 – Recent score

health is also decreased by 1 each time it's a wrong matching.

Picture 36 – Health

After all, set *row1 = col1 = row2 = col2 = -1*, which means that *row1*,*col1*,*row2*,*col2* don't hold the index of any elements in the matrix *a*, and the program will update the recent *tempPoint*, *coefficient*, *numOfHints*, *health* to the right-side board.

While playing, whenever user presses *Tab* key, *hint* function (section 2.6.) will be called, it randomly shows a pair of elements that can be matched, *thanhSang* function (section 2.2.3.)will colorize two boxes containing these two elements into pink color, a funny sound is played, and the color will get back to the first color after a while.



Picture 37 - Hint

numOfHints will be decreased by 1 each time *Tab* pressing, and the program will update *numOfHints* on the board to the right-side board.

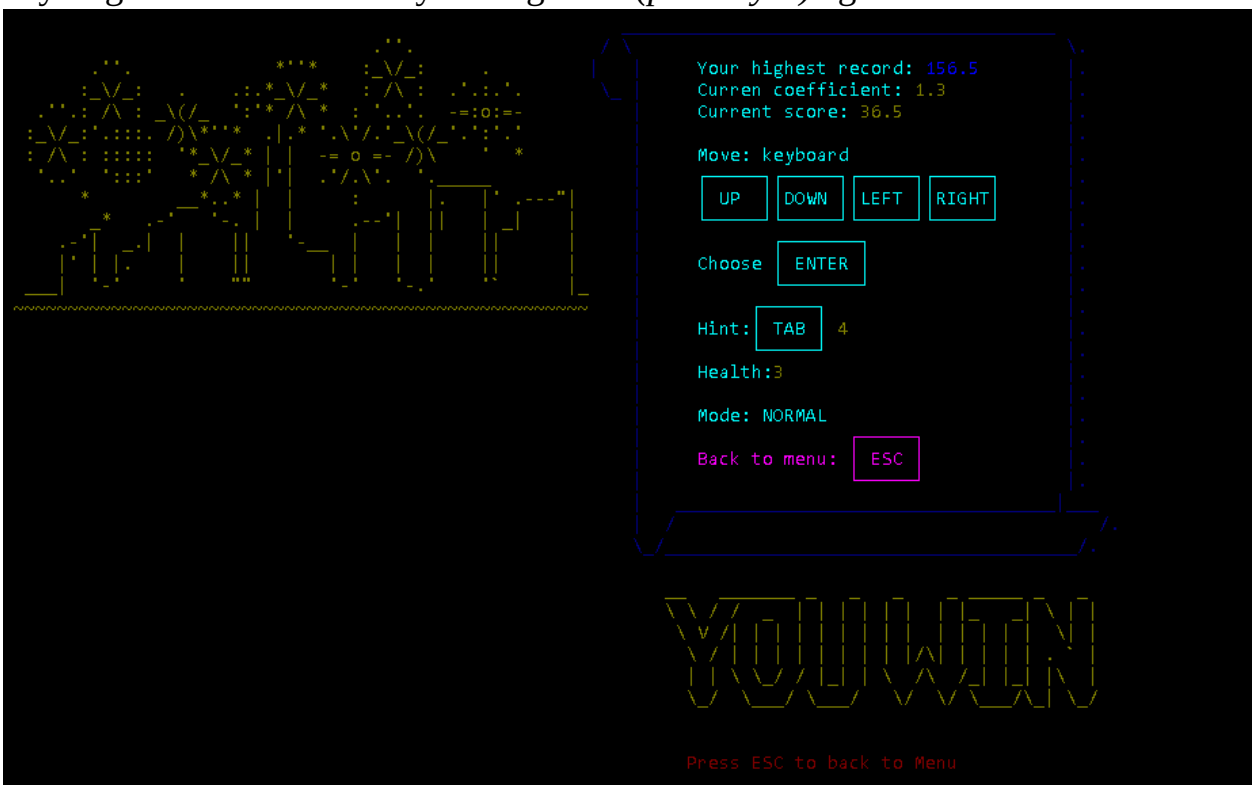
Picture 38 – numOfHints decrease

While playing, the program will repeatedly call *checkExistPair* function to check whether there's still any pair of elements can be matched or not. If the *checkExistPair* function returns *false* value, it means that there's no pair of elements can be matched, and
If *numOfHints* reach 0, when pressing *Tab*, there will be nothing happen.
While playing, if user presses *ESC* key, the program will call *menu(posPlayer)*, and all points won't be saved.
The game will stop in 2 situations:
If user's *health* reaches 0, it means that the game hasn't been successfully completed. The program calls *saveRecords* function to save the *tempPoint* if it is greater than the highest score, and *failGame* function (section 2.9.) to play a sad sound while printing a notification

showing that you lost the game. After all, user will asked to press *ESC* key to get back to MENU by calling *menu(posPlayer)* again.

Picture 39 – Lose game

If *numOfPairs* reaches 0, it means that the game has been successfully completed. The program calls *saveRecords* function to save the *tempPoint* if it is greater than the highest score, and *winGame* function (section 2.8.) to play a cheerful sound while printing a notification showing that you won the game. After that, user will asked to press *ESC* key to get back to MENU by calling *menu(posPlayer)* again.



Picture 40 – Win game

But if the *mode* is “STEP BY STEP” mode, the game won’t stop until finishing all stages. At that time, the game will have 5 stages (section 2.7.).

1.19. *Save Records*

saveRecords(profile player, int numOfPlayer, int posPlayer, double tempPoint, int mode)*

If *mode* is NORMAL (*mode == 0)*, it will make a comparison between *tempPoint* and *player[posPlayer].normalScore* (section 1.20), if *tempPoint* is greater than *player[posPlayer].normalScore*, then it will update the value of *tempPoint* to *player[posPlayer].normalScore*.

If *mode* is HIDDEN *(mode == 1)*, it will make a comparison between *tempPoint* and *player[posPlayer].hiddenScore* (section 1.20), if *tempPoint* is greater than *player[posPlayer].hiddenScore*, then it will update the value of *tempPoint* to *player[posPlayer].hiddenScore*.

If *mode* is STEP BY STEP *(mode == 2)*, it will make a comparison between *tempPoint* and *player[posPlayer].stepScore* (section 1.20), if *tempPoint* is greater than *player[posPlayer].stepScore*, then it will update the value of *tempPoint* to *player[posPlayer].stepScore*. After all, the program opens the “records.txt” file in writing mode and writes the file again.

1.20. *Structure*

We used #ifndef STRUCT_H_ and #define STRUCT_H_ , and after having created a structure, we used #endif

We have to use them so as to prevent multiple inclusions of the same header file, which can lead to compilation errors due to redefinition of types, functions, or variables.

We created a *struct* named *profile* with two *string* variables *username*, *password*, three *double* variables *normalScore*, *hiddenScore*, *stepScore*.

2. *Advanced Features*

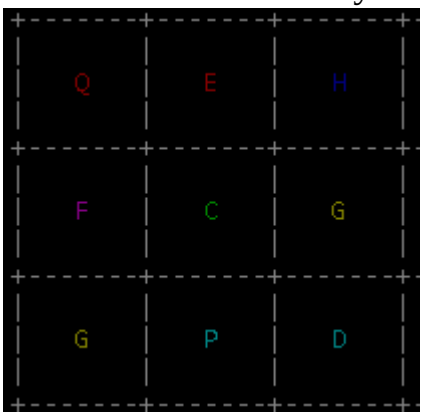
2.1. *Color Effects*

2.1.1. *Drawing Box*

box(int x, int y, int height, int width, char value, int mode, int colorBox)

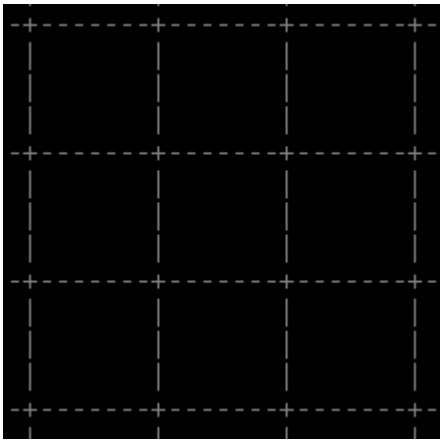
Reference: *https://www.youtube.com/watch?v=9gWIbhOudEk*

This function is used to draw the boxes surrounding the elements of the matrix.
Assume that the monitor is a coordinate system with *x* and *y* axis
X plays for the coordinate of horizontal axis, while *y* plays the role of the coordinate of vertical axis. *Height* and *width* will determine the height and width of the box when it is drawn, and an *int* variable *colorBox* to determine which color it will use.
Firstly, the function will call function *gotoxy* (section 2.1.3.) to get access to the coordinates (determined by programmer) to draw a box filled with ‘ ‘ character, then, it will come to the coordinates of the rims of the box, then it will change the ‘ ‘ value into another values as a “wall” among the elements. After all, it will come to the middle space in the box to write the value of the element, depends on which *mode* it is.
If *mode* is 0, which means that it’s NORMAL *mode*, then, the value in the middle will be colorized randomly depends on the ASCII value of the character by using *setColor* function (section 2.1.2.).



Picture 41 – Color effect (Normal)

Otherwise, *mode* is equal to 1, it means that *mode* is HIDDEN, then, the value will be hidden. In other words, the color will be set to 0, the same as the color of the console.



Picture 42 – Color effect (Hidden)

There are two more drawing box functions:
void box2(int x, int y, int width, int high, int color, string value)
void box3(int x, int y, int width, int high, string value)
Box function is used for the game display, while *box2* draws the boxes in *menu* and *login* screen

Picture 43 – Draw Menu choices

box3 draws the boxes containing the arrow keys in the right-side board while playing game.

Picture 44 – Drawing in score board

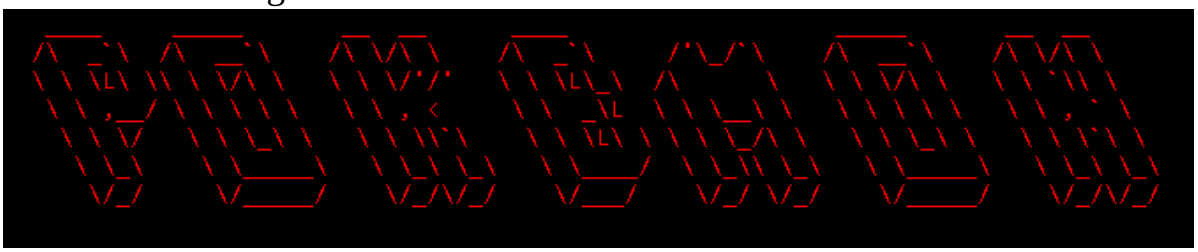
2.1.2. *Text Colorizing*

void setColor(int color)

Reference: *https://www.youtube.com/watch?v=9gWIbhOudEk*

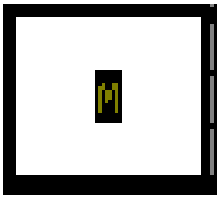
This function is used to set the color depends on the *int* variable *color* that got from programmer. *Color* has some values that are

used for colorizing the texts



Picture 45 – Text colorizing

while some different ones are used for colorizing the background.



Picture 46 – Pointer colorizing

2.1.3. *goToXY Function*

void gotoxy(short x, short y)

Reference: *https://www.youtube.com/watch?v=9gWIbhOudEk*

This function is used to move the console cursor to a determined coordinates *(x , y)*.

2.2. *Visual Effects*

2.2.1. *Resizing Console*

void resizeConsole(int width, int height)

Reference: *https://www.youtube.com/watch?v=9gWIbhOudEk*

resizeConsole function is used to change the size of the console screen for aesthetics and to avoid encountering errors while drawing the game.

Using *resizeConsole* function, we could change the size of the console screen to fit the contents being shown.

2.2.2. *Show / Hide Cursor*

void ShowCur(bool CursorVisibility)

Reference: *https://daynhauhoc.com/t/hoi-ve-ham-showcur-trong-c/87106*

This function is used to hide or show the console cursor. As our game will have some parts that don't need to show the console

cursor, for aesthetics, we have to use this function. The console cursor will be hidden when we put 0 to *CursorVisibility*, as 0 is similar to *false* value. Meanwhile, putting 1 to *CursorVisibility* will help to show the console cursor since 1 is equivalent to *true* value.

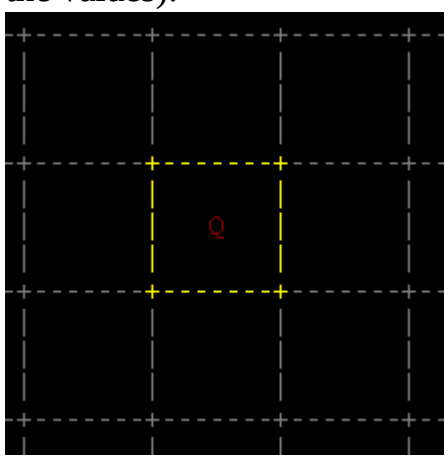
2.2.3. *Pointer Colorizing*

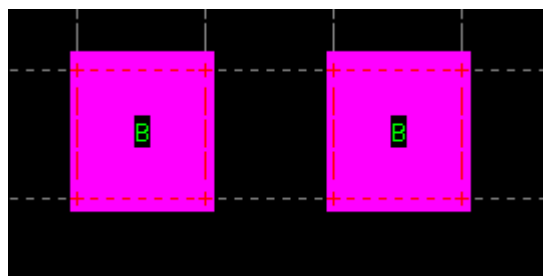
void thanhSang(int x, int y, int w, int h, int b_color, char value, int mode)

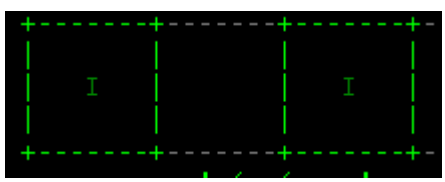
Reference: *https://www.youtube.com/watch?v=9gWIbhOudEk*

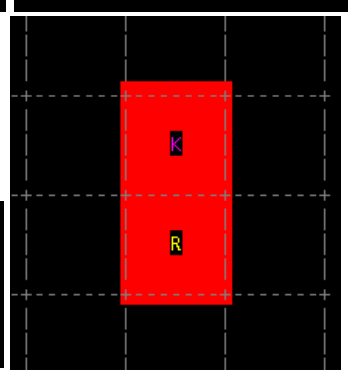
This function is used to colorize the boxes in the game play. When using arrow keys to move or pressing *Enter* key or *Tab* key, *thanhSang* will help to change the color at that position.

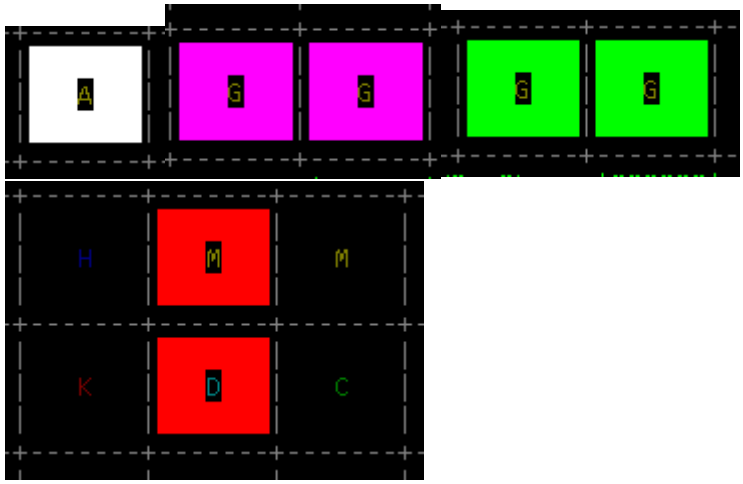
If it is in the HIDDEN *mode*, it will colorize the frame of the box when moving and show the value inside in a determined color, while other values will be hidden (the colors of these elements are the same as the background's, so that player won't be able to see the values).











Picture 47 – Pointer colorizing

2.3. *Sound Effects*

PlaySound(TEXT(filename, NULL, SND_FILENAME | SND_ASYNC | SND_LOOP);
PlaySound(TEXT(filename, NULL, SND_FILENAME)
PlaySound(TEXT(filename, NULL, SND_FILENAME | SND_ASYNC))
PlaySoundA(NULL, NULL, SND_FILENAME)

Reference: *https://www.youtube.com/watch?v=Y_8DhMN_wHE*

These functions are used to play sounds while playing game. *filename* is the name of the sound file saved in the computer. Using *SND_FILENAME* is flag that specifies the type of sound to be played. We used ".wav" sound files when using these functions. Meanwhile, *SND_ASYNC* helps the sound be played asynchronously, meaning the program won't block the execution of the following codes. *SND_LOOP* is used for playing the sound in a continuous loop until the program reaches another sound file or when user quits the program. *PlaySoundA* function is just used to stop our background music.

Our sounds and background music are from the Internet.

Background music:

Reference:
https://www.youtube.com/watch?v=Hvdfx9avekU&list=PLwJjxqYuirCLkq42mGw4XKGQlpZSfxsYd&index=5

Sounds:

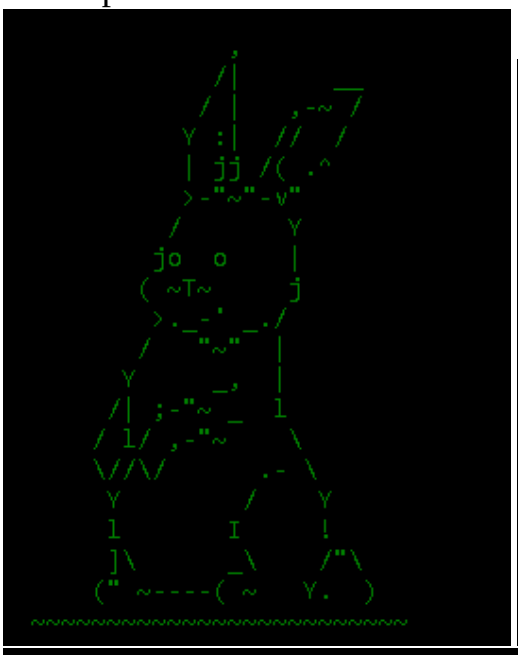
Reference: *https://mixkit.co/*

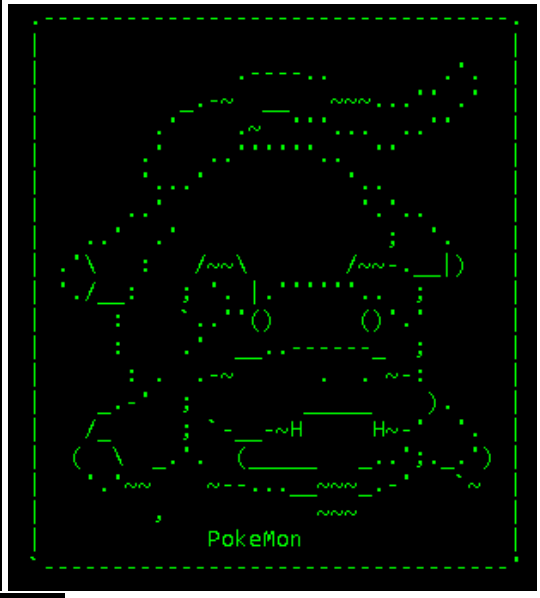
2.4. *Background*

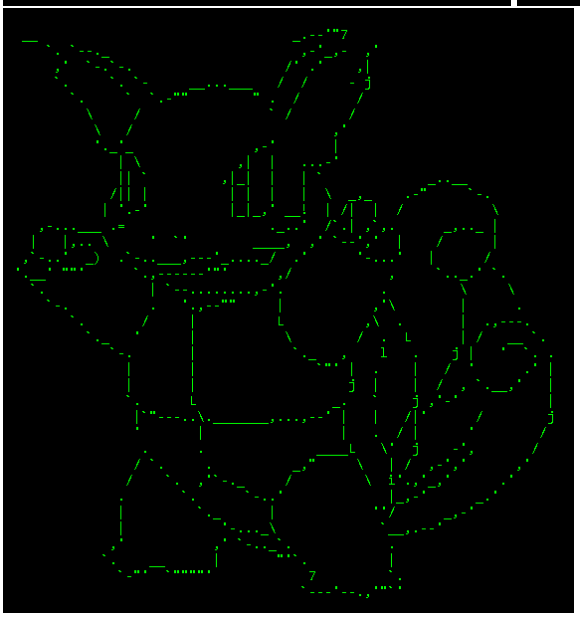
void backgroundPlay(int index)

Calling *backgroundPlay* function will draw a background while playing game based on the size of the matrix. We created several backgrounds in text files and the function will read the contents inside one of the files to draw them onto the monitor.

These pictures are some of our backgrounds:







Picture 48 - Backgrounds

Reference: *https://www.asciiart.eu/video-games/pokemon*

2.5. *Leaderboard*

void leaderboard(profile player, int numOfPlayer, int posPlayer)*

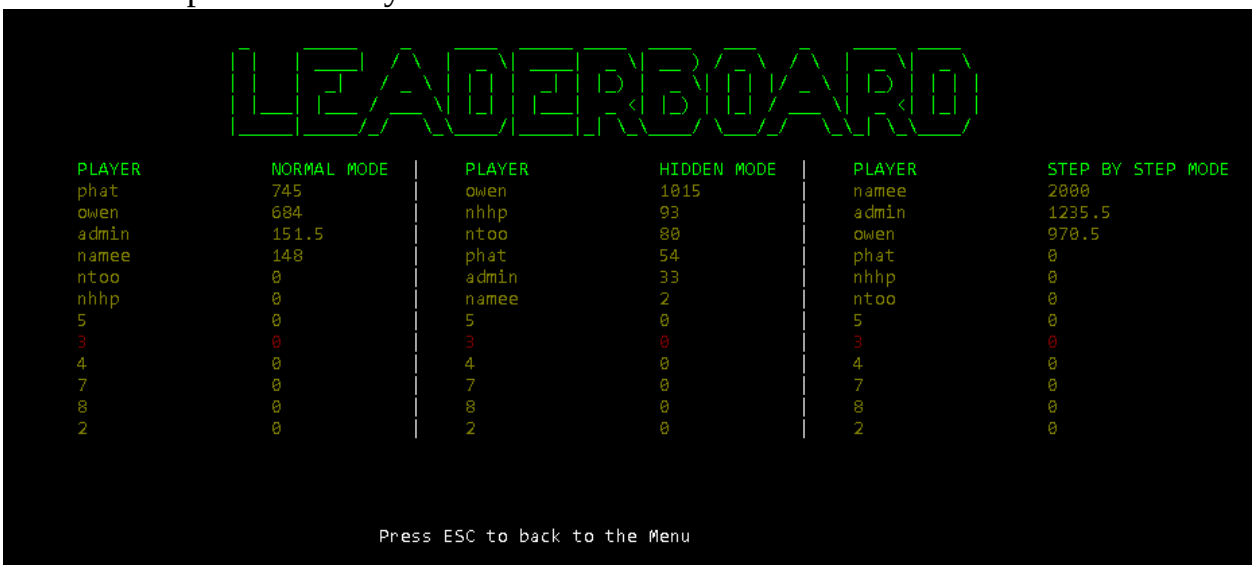
When user presses the *Enter* key in the MENU screen at the "RANK" box, the program will call the *leaderboard* function to show the ranks of players in our game.

Players' ranks will be arranged in ascending order, and the name and points of recent user will be shown in distinguished color.

There will be three leaderboards, one of the NORMAL mode, one in HIDDEN mode, and the other in STEP BY STEP mode. We decided to separate the points at these three modes that will help users fairly ranked.

The function will generate a new array which is the same as *player* for arranging. It will move the position of the player whose points are greater onto the higher place in the ranking board. The points in NORMAL mode, HIDDEN mode, and STEP BY STEP mode are independent, as we had created three *double* variables *normalScore*, *hiddenScore*, *stepScore* in the structure *profile*.

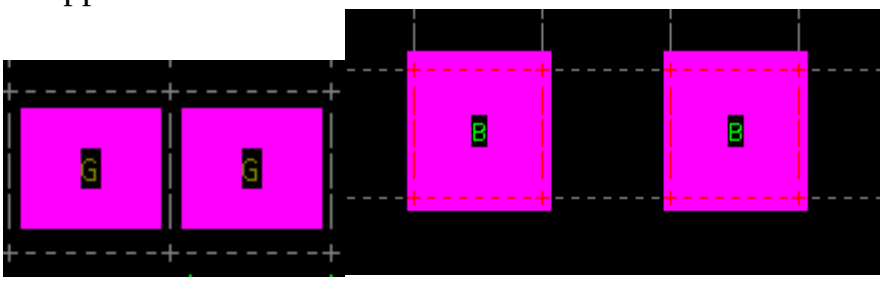
After all, the program will display three leaderboards to the monitor and asks user to press *ESC* key to leave the leaderboard zone.



Picture 49 – Leaderboard

2.6. *Move Suggestion*

*void hint(char** a, int p, int q, int mode)*
We created four *int* variables *posRow1*, *posCol1*, *posRow2*, *posCol2* to hold the index of two elements. Then, we use *rand()* function to random the value of these four variables in a *while(true)* loop, Each time they get a value, the program calls *checkAll(a, p, q, posRow1, posCol1, posRow2, posCol2)* to check whether the two elements can be matched or not. It will break the *while* loop when *checkAll* function returns *true* value, which means it finds one pair of element can be matched in the matrix *a*. Then, the *thanhSang* function will be called to colorize these two elements to show the positions of them to user, the color will disappear after a while.



Picture 50 – Hint

2.7. *Stage Difficulty Increase*

We created three modes in our game. The easiest mode is "NORMAL MODE", which is the basic mode of each matching game. Besides, "HIDDEN MODE" and "STEP BY STEP" are the two new modes that we created to make our game more interesting.
In "HIDDEN MODE", the characters inside the boxes will be hidden, and user will have to remember the positions while playing.
Moreover, in "STEP BY STEP" mode, user will be asked to play multiple stages. There will be 5 stages:
Stage 1: 4x4 NORMAL MODE puzzle
Stage 2: 6x6 NORMAL MODE puzzle
Stage 3: 8x8 NORMAL MODE puzzle
Stage 4: 4x4 HIDDEN MODE puzzle
Stage 5: 6x6 HIDDEN MODE puzzle
After finishing each stage, your current points and coefficient will be saved for the next stage.

2.8. *Game Finish Verify (Win)*

*void winGame(int numOfCols, profile *player, int numOfPlayer, int posPlayer)*

This function will be called when user won the game, it will read a file to show the notification that congratulate the winning of user, whereas a cheerful sound will be played at the same time. After all, the program will ask user to press *ESC* key to get back to the MENU screen by calling *menu(posPlayer)*.

Picture 51 – Win game

2.9. *Game Finish Verify (Lose)*

*void failGame(int numOfCols, profile *player, int numOfPlayer, int posPlayer)*

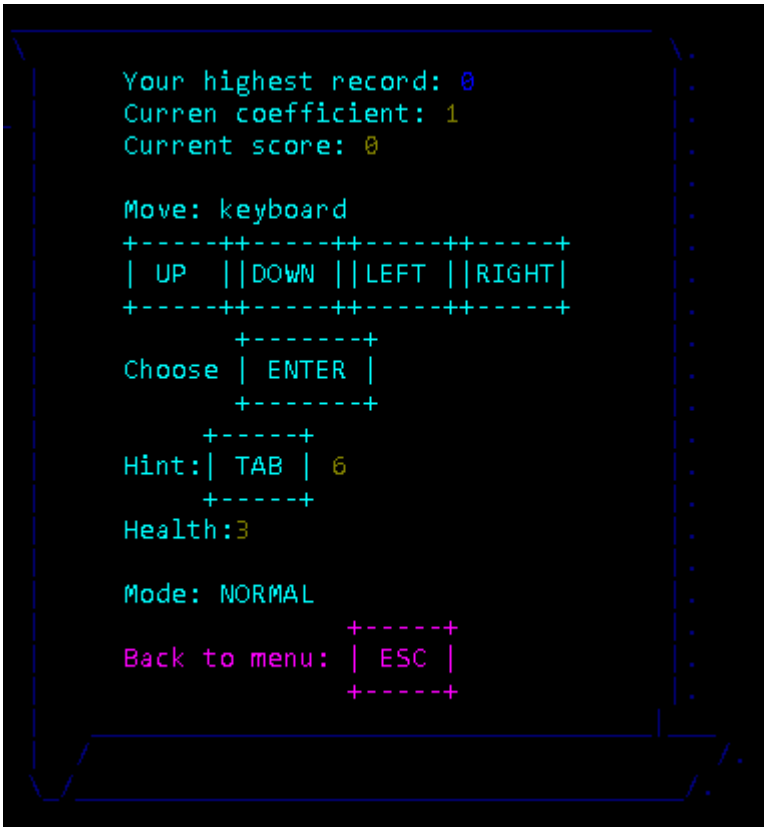
This function will be called when user lost the game, it will read a file to show the notification as a condolence to user, whereas a sad sound will be played at the same time. After all, the program will ask user to press ESC key to get back to the MENU screen by calling *menu(posPlayer)*.

Picture 52 – Lose game

2.10. *Score Board*

void boardScore(int p, int q, double normalScore, double hiddenScore, int mode)

This function is used to draw the right-side board while playing game containing some instructions, user's points, …

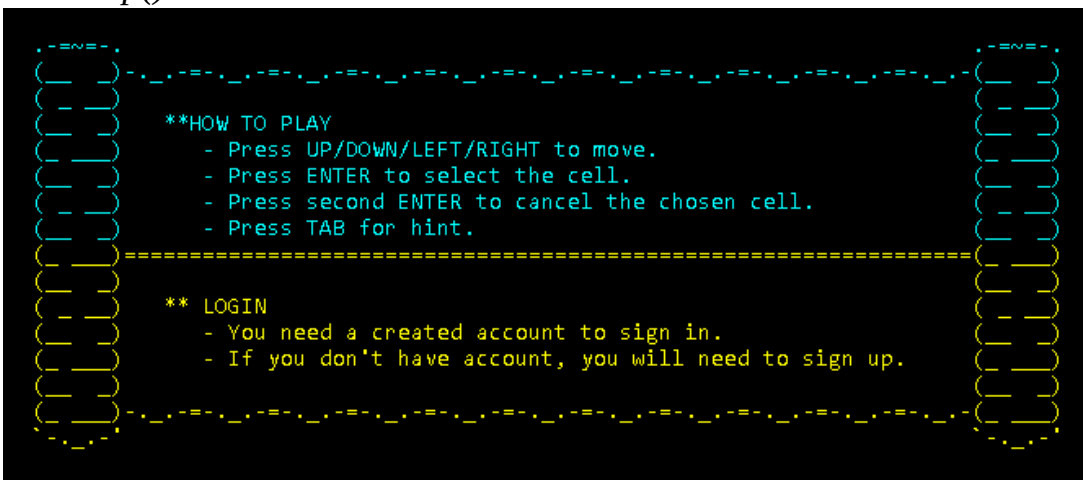


Picture 53 – Score board

Reference: *https://www.asciiart.eu/art-and-design/borders*

2.11. *Help*

void help()



Picture 54 – Help

Reference: *https://www.asciiart.eu/art-and-design/borders*

This function is the first one to be called in the game. It will show some instructions to play our game

3. *Other Features*

3.1. *Forgot Password*

void forgot()

Reference:

https://www.youtube.com/watch?v=yo3ImzEwP50&list=PLVUI3PRPBHxU8IAvLI5yfOUtkI9Mwndgg&index=2

Sometimes, user may forgot his/her account's password, when it's in *start()* function, user is able to press the *Enter* key to choose "Forgot password", then, the program will call *forgot()* function for helping.

It will give user two choices:

Entering number 1 to find the password on the condition that they remember their account's name. Otherwise, there's no way to get your password.

```
You forgot the password? No worries

Press 1  || search your id by username
Press 2  || go back the main menu

Enter your choice : 1

Enter the username which you remembered: 123
```

```
  Your username is found!
 ===> Your password is: 1233
```

Picture 55 – Find password

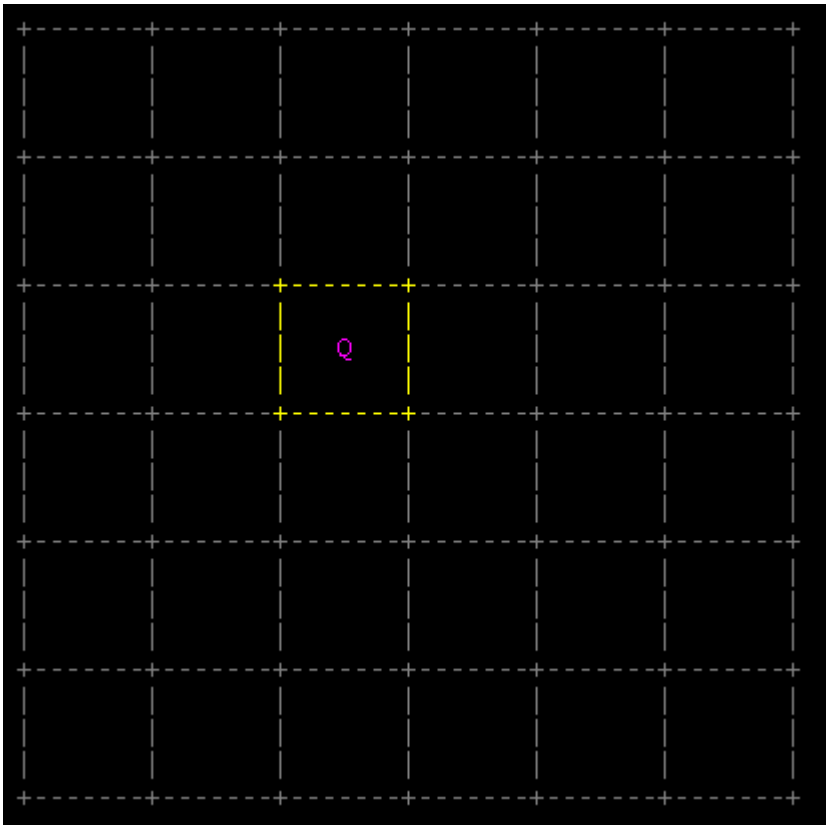
The other choice is entering number 2, it's when user suddenly remember the password of the account, and entering number 2 will help to call *start()* again.
If user enters a number that is different from 1 and 2, then, program will print a notification to the monitor showing that it's a wrong choice and call *forgot()* function again.

```
You forgot the password? No worries

Press 1  || search your id by username
Press 2  || go back the main menu

Enter your choice : 3
        Wrong choice! Please try again
```

Picture 56 – Wrong choice

3.2. *Hidden Mode*

HIDDEN mode is one of our game modes, it will increase the difficulties of the game. Since when user choose HIDDEN mode at the MENU part, the program will change the way to display the game instead of traditionally showing all the elements. The program now knows that it needs to hide the elements. As a result, when user completed the game in HIDDEN mode, points will be saved to *hiddenScore* stored in the user's account.



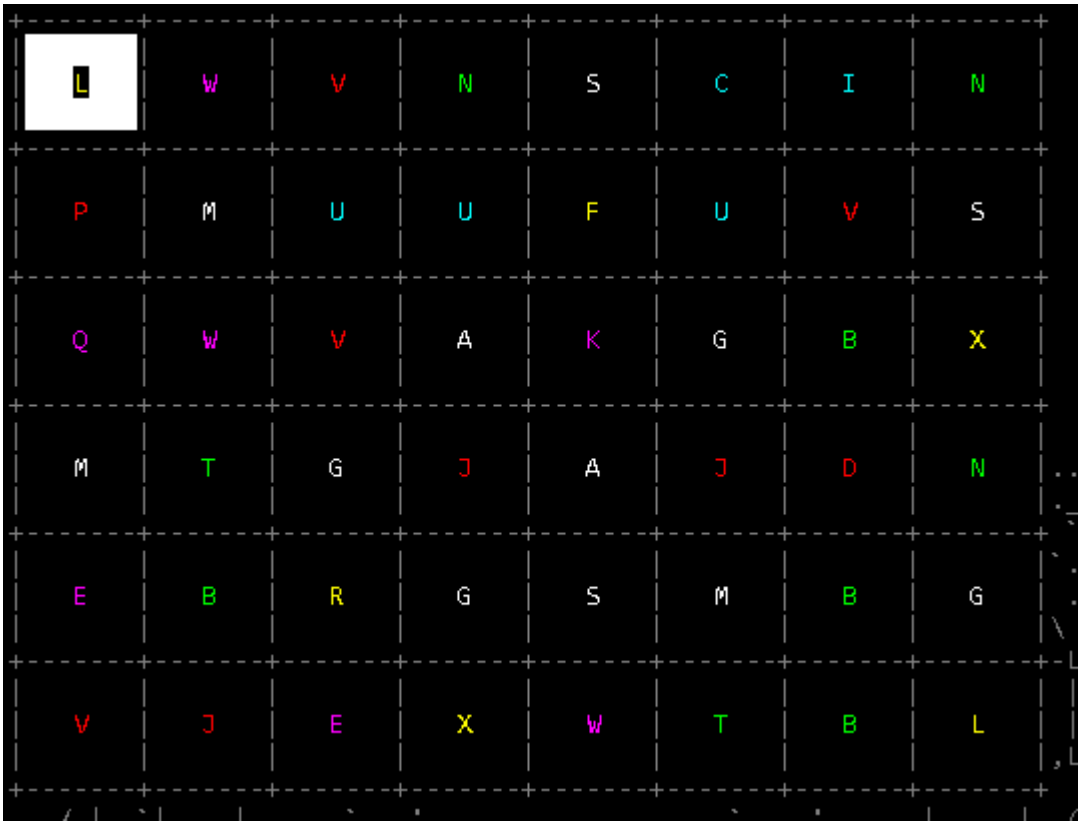
Picture 57 – Hidden mode

3.3. *Custom Level*

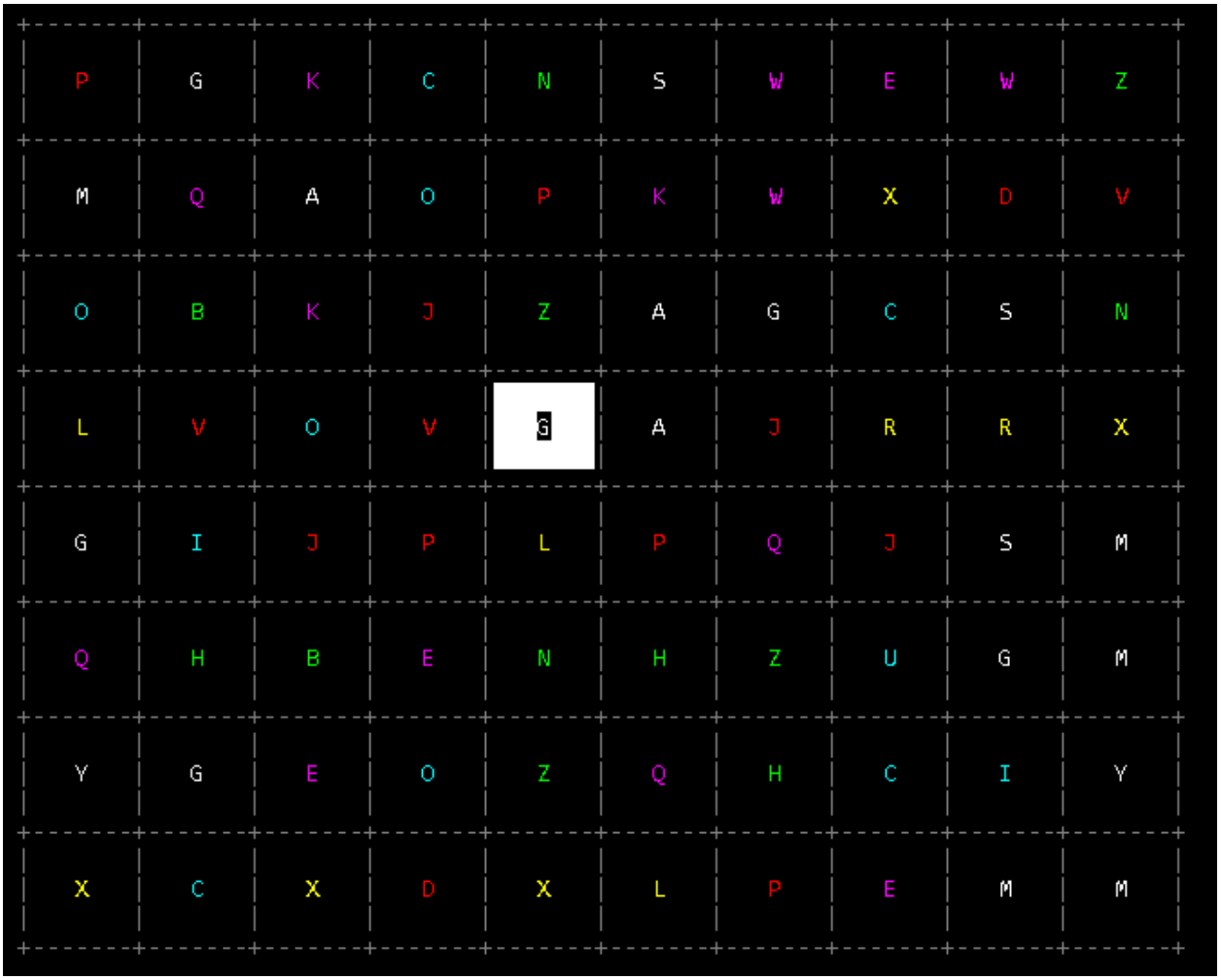
Custom level is a level that will let user create the puzzle of any size according to his / her wanting. We just accept the number of rows and columns which is from 4 to 10 due to the fact that we don't want our game to be too small or too large.

These are some customized puzzles:

NORMAL MODE

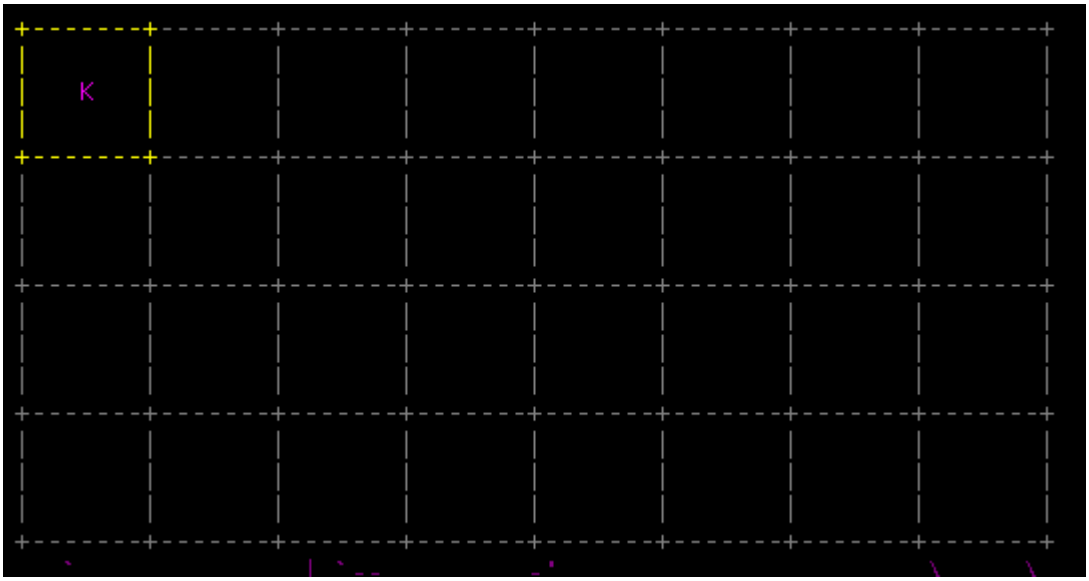


Picture 58 – Normal puzzle

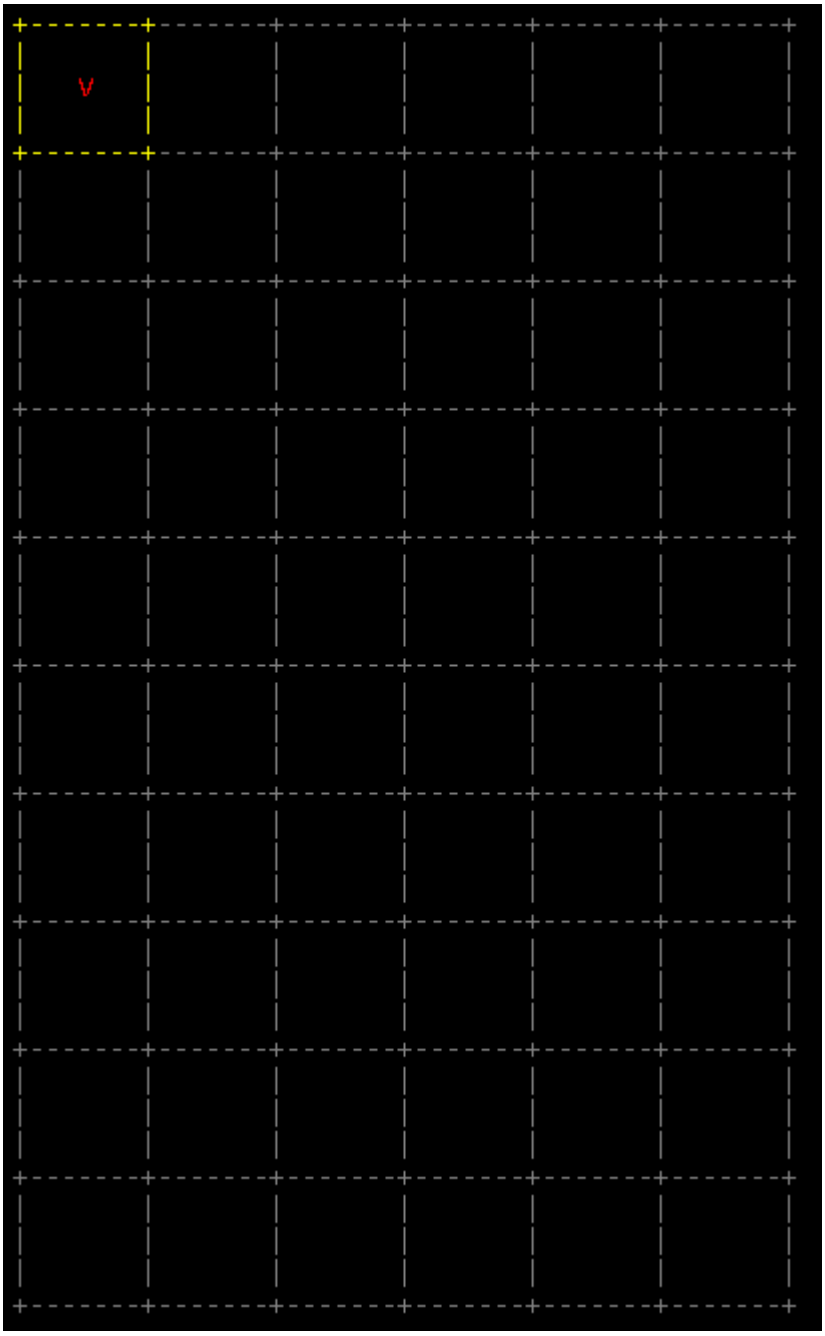


Picture 59 – Normal puzzle

HIDDEN MODE



Picture 60 – Hidden puzzle



Picture 61 – Hidden puzzle

VI. *Demonstration Video*

https://www.youtube.com/watch?v=IPFroBZufyk

VII. *Overall*

Finally, our Pikachu Game project is successfully done after having been supported by our instructor. While the working process, we gradually realized that this project helped us to know how to be responsible for our works. We also got lots of experiences after this project that we knew the importance of analyzing and solving problems

However, our project can still have some mistakes due to our lack of experience. If there are any shortcomings in our project, honestly, we hope to get feedbacks in order to fix them, and that will be a huge experience for us to improve our qualification.

Hope you enjoy our game!

VIII. *Table Of Figures*

Picture 1 - Executing....6
Picture 2 - Executing....7
Picture 3 - Executing....7
Picture 4 - Executing....8
Picture 5 - Executing....8
Picture 6 - Login....9
Picture 7 - Menu....10
Picture 8 - Play....11
Picture 9 - Start....15
Picture 10 – Login successfully....16
Picture 11 – Login unsuccessfully....17
Picture 12 – Register unsuccessfully....17
Picture 13 – Register successfully....17
Picture 14 – Menu screen....18
Picture 15 – Choose game mode....19
Picture 16 – Choose level....20
Picture 17 – Normal Standard....20
Picture 18 – Custom....21
Picture 19 – Normal Custom....22
Picture 20 - Hidden....22
Picture 21 – Hidden Custom....23
Picture 22 - Introduction....24
Picture 23 – I Matching....26
Picture 24 – I Matching....26
Picture 25 – L Matching....28
Picture 26 – Upper U Matching....29
Picture 27 – Right U Matching....30
Picture 28 – Left U Matching....30
Picture 29 – Down U Matching....31
Picture 30 – Z Matching....31
Picture 31 – Choose element....36
Picture 32 – Cancel chosen element....36
Picture 33 – Correct matching....36
Picture 34 – Wrong matching....37
Picture 35 – Recent score....37
Picture 36 – Health....37
Picture 37 - Hint....38

Picture 38 – numOfHints decrease 38
Picture 39 – Lose game.....39
Picture 40 – Win game.....39
Picture 41 – Color effect (Normal) 41
Picture 42 – Color effect (Hidden).....42
Picture 43 – Draw Menu choices 42
Picture 44 – Drawing in score board.....42
Picture 45 – Text colorizing.....43
Picture 46 – Pointer colorizing 43
Picture 47 – Pointer colorizing 45
Picture 48 - Backgrounds.....47
Picture 49 – Leaderboard 47
Picture 50 – Hint 48
Picture 51 – Win game.....49
Picture 52 – Lose game.....49
Picture 53 – Score board 50
Picture 54 – Help.....51
Picture 55 – Find password.....52
Picture 56 – Wrong choice.....52
Picture 57 – Hidden mode.....53
Picture 58 – Normal puzzle.....54
Picture 59 – Normal puzzle.....55
Picture 60 – Hidden puzzle 55
Picture 61 – Hidden puzzle 56

IX. *References*

Logo HCMUS:
https://www.facebook.com/VNUHCM.US?mibextid=ZbWKwL

How to setup MinGW file and .exe file:
Lê Nguyên Nhật – 23127239 – 23CLC10

https://www.youtube.com/watch?v=yo3ImzEwP50&list=PLVUI3PRPBHxU8IAvLI5yfOUtkI9Mwndgg&index=2
https://github.com/PhVanMin/Pikachuuu/blob/master/Check1.cpp
https://www.youtube.com/watch?v=9gWIbhOudEk
https://daynhauhoc.com/t/hoi-ve-ham-showcur-trong-c/87106
https://www.youtube.com/watch?v=Y_8DhMN_wHE
https://www.youtube.com/watch?v=Hvdfx9avekU&list=PLwJjxqYuirCLkq42mGw4XKGQlpZSfxsYd&index=5
https://mixkit.co/
https://www.asciiart.eu/video-games/pokemon
https://www.asciiart.eu/art-and-design/borders